

Full length article

# Automatic lightweight networks for real-time road crack detection with DPSO

Guijie Zhu<sup>a</sup>, Shui-Long Shen<sup>a,1,\*</sup>, Junjie Yao<sup>d</sup>, Meihua Wang<sup>c</sup>, Jiafan Zhuang<sup>b</sup>, Zhun Fan<sup>d,\*</sup>

<sup>a</sup> Guangdong Engineering Research Center for Smart Construction and Maintenance of Infrastructures of Coastal Cities, Department of Civil Engineering and Smart Cities, College of Engineering, Shantou University, Shantou, Guangdong 515063, China

<sup>b</sup> MOE Key Laboratory of Intelligent Manufacturing Technology & Department of Electronic Engineering, College of Engineering, Shantou University, Shantou, Guangdong 515063, China

<sup>c</sup> College of Mathematics and Informatics, South China Agricultural University, Guangzhou, Guangdong 510642, China

<sup>d</sup> Shenzhen Institute of Advanced Study, University of Electronic Science and Technology of China, Shenzhen 518000, China

## ARTICLE INFO

### Keywords:

Automatic lightweight networks  
Evolutionary neural architecture search (ENAS)  
Discrete particle swarm optimization (DPSO)  
Encoder-decoder network  
Road crack segmentation  
Real-time detection

## ABSTRACT

This study proposes a framework for automatic designing lightweight neural networks optimized for real-time road crack detection. This method overcomes the limitations of manual design and high computational complexity in existing networks. Building on a U-shaped encoder-decoder architecture, a scalable search space is designed by integrating attention modules and residual connections. Then, a discrete particle swarm optimization algorithm is employed to automatically discover high-performance networks in the search space and to investigate the possibility of evolving superior lightweight models with fewer convolutional channels. Experimental results on four datasets, i.e., CrackForest, CRKWH100, Combined Crack Dataset (CCD), and SBGCrack, demonstrate that the discovered lightweight network named AutoCrackNet achieves superior performance with F1 of 92.22 %, 95.98 %, 95.35 %, and 90.54 %, and IoU of 88.66 %, 92.32 %, 91.54 %, and 83.89 %, respectively. AutoCrackNet has only 0.043 M parameters, which is significantly fewer than existing models. Ablation studies validate the effectiveness of integrating attention modules and residual connections. Furthermore, when deployed on a Jetson Xavier NX-equipped UAV, AutoCrackNet achieves real-time crack detection at 29 Frames Per Second (FPS), showcasing its potential for practical infrastructure applications.

## 1. Introduction

Cracks are a common type of surface damage in infrastructure. Accurate and efficient crack identification is crucial for ensuring the structural safety and longevity of civil infrastructure. Traditional manual inspection is not only labor-intensive and subjective but also prone to errors and inefficiencies [1]. While early computer vision-based methods (e.g., edge detection [2], thresholding [3,4], morphological operation [5]) showed promise for simple crack images, their sensitivity to parameter settings and poor adaptability to complex backgrounds limit their practical utility.

In contrast, automated crack detection based on deep learning has shown promising accuracy and advantages in processing crack images with complex background. In recent years, U-Net [6] and its variants [7–10] have emerged as the predominant frameworks for crack

segmentation, owing to their effective encoder-decoder structure with skip connections. Recent derived models [10,11] integrate attention mechanisms to enhance crack detection accuracy. However, the backbone structures of U-Net and its variants are often constructed by repeatedly stacking identical components. This design pattern may not be optimal, as it could limit the model's capacity for feature extraction and representation. Moreover, these models are manually designed and heavily rely on expert knowledge, particularly in terms of network architecture and parameter configuration. To improve detection accuracy, they typically involve a large number of parameters and high computational complexity, making them difficult to deploy on mobile devices and thereby limiting their practical application.

In recent years, to accelerate the practical application of models on mobile devices such as drones and smartphones, lightweight networks for crack detection have garnered increasing attention. Techniques like

\* Corresponding authors.

E-mail addresses: [shensl@stu.edu.cn](mailto:shensl@stu.edu.cn) (S.-L. Shen), [fanzhun@uestc.edu.cn](mailto:fanzhun@uestc.edu.cn) (Z. Fan).

<sup>1</sup> ORCID: 0000-0002-5610-7988.

model pruning [12], knowledge distillation [13], and depthwise separable convolution [14] have been introduced to reduce model complexity and enable real-time crack detection. Yet these approaches still require manual optimization and often sacrifice detection accuracy in complex scenarios.

Recently, Neural Architecture Search (NAS), particularly Evolutionary NAS (ENAS), has emerged as a promising paradigm to automate neural networks design [15]. Various optimization algorithms, including Genetic Algorithms (GA) [16], Particle Swarm Optimization (PSO) [17], Chicken Swarm Optimization [18], and Greylag Goose Optimization [19], have been successfully applied to NAS. These methods have demonstrated effectiveness in tasks such as image classification [16], object detection [18,20], semantic segmentation [21,22], and even in broader domains like sustainable energy management [23]. Among them, PSO stands out due to its strong global search capability and fast convergence. PSO has shown notable success in both architecture optimization [24] and control system design [25–27]. For example, Gong and Ma et al. [24] utilized PSO to optimize a lightweight network for object detection. Al-Jodah et al. [26] used PSO to optimize a neural network PID controller, enhancing the stability and navigation of an omnidirectional robot. Despite its potential, no prior work has specifically explored PSO-based architecture search for pixel-level road crack detection. This gap motivates our work.

To sum up, existing methods suffer from two main limitations: (1) most are manually designed and heavily rely on expert knowledge, which results in a time-consuming and inefficient design process that struggles to yield optimal models for complex real-world environments; (2) many approaches tend to focus either on improving accuracy or on reducing model complexity, but rarely achieve a satisfactory balance between the two, which limits their applicability in practical scenarios.

To address these issues, we propose a PSO-NAS framework that combines the strengths of PSO and NAS to automatically design an efficient and accurate model for road crack detection. Inspired by the U-Net architecture, we first construct a scalable search space and then employ a discrete PSO (DPSO) algorithm to automatically optimize network architectures within this space. The search space is designed with two key considerations: (1) incorporating attention mechanisms and residual connections into the building blocks to enhance the model's representational capacity and detection accuracy; and (2) constraining the number of convolutional channels in each block to reduce parameter size and computational cost. These design strategies are instrumental in achieving a well-balanced trade-off between detection accuracy and model efficiency.

In summary, the key contributions of this work are as follows:

(1) An approach is proposed for automatic designing lightweight networks specifically for road crack detection. This method combines a U-shaped architecture with a DPSO-driven search strategy to generate a lightweight model, named AutoCrackNet, which outperforms existing networks.

(2) Based on a U-shaped architecture, an extensible search space is devised to enable the discovery of lightweight models maintaining accuracy-complexity equilibrium under constrained channel configurations.

(3) A FTCrack dataset is constructed, which comprises 65 floor tile crack images captured under diverse lighting conditions using a smartphone. These images are used as test samples to verify the proposed AutoCrackNet's cross-dataset generalization.

(4) Extensive experiments on four datasets demonstrate that AutoCrackNet achieves state-of-the-art performance with only 0.043 M parameters. Furthermore, it performs real-time crack detection at 29 FPS on a UAV equipped with an embedded device, which confirms its practical applicability.

The reminder of this paper is organized as follows. Section 2 provides a literature review on related work. Section 3 presents a detailed description of the proposed methodology. Section 4 presents a detailed description of experiments. Section 5 shows the results and model

performance. Section 6 presents a discussion of the study's strengths and limitations and proposes potential directions for future research. The final section summarizes the findings.

## 2. Related work

### 2.1. DCNN-based crack detection

With the advancement of artificial intelligence, machine learning and deep learning have been widely applied in fields such as agriculture, medicine, and transportation. They have shown strong performance in tasks like crop disease detection [28], yield prediction [29], groundwater resource estimation [30], retinal vessel segmentation [22], and road damage detection [31–33]. However, in road crack detection, traditional machine learning methods (e.g., support vector machine [34] and random forests [35]) rely heavily on manual feature engineering, making them less effective under complex backgrounds. In contrast, deep convolutional neural networks (DCNNs) [36–39] automatically learn hierarchical features, offering superior performance in crack detection tasks.

Recent research has introduced various DCNN-based architectures for crack detection, which can be broadly classified into seven categories:

1) *CNNs*: For instance, Fan et al. [31] proposed an ensemble of three parallel CNNs for pixel-wise pavement crack detection. Zhang and his collaborators proposed a series of CNN models, including CrackNet [40], CrackNetII [41], and CrackNetV [42], for 3D asphalt surface crack detection.

2) *U-Net*: Fan et al. [43] proposed U-HDN, which integrates multi-dilation and hierarchical learning modules into U-Net to improve accuracy. Yao et al. [10] designed CrackResU-Net by incorporating a pyramid region attention module into U-Net.

3) *DeepLab*: Ji et al. [44] applied DeepLabv3+ for pixel-level asphalt crack segmentation, while Sun et al. [45] enhanced it with a multi-scale attention block to improve accuracy.

4) *SegNet* [46]: For instance, Chen et al. [47] used standard SegNet for pavement and bridge cracks. Li et al. [48] proposed OUR-Net by enhancing SegNet with Octave convolution and Octave Max Unpooling for pavement crack segmentation.

5) *Mask R-CNN* [49]: Originally designed for instance segmentation, Mask R-CNN has been adapted for crack detection. Long et al. [50] applied it to pixel-level crack identification, and Liu et al. [51] introduced improvements such as an enhanced feature pyramid network and ResNet101 backbone for detecting fine cracks.

6) *Transformer* [52]: Transformer has recently gained attention in crack detection due to their ability to capture long-range dependencies. Wang and Su [53] used a multi-block transformer to model multi-scale features of concrete cracks. Guo et al. [54] developed an attention-enhanced transformer for pixel-accurate pavement crack segmentation.

7) *Hybrid architecture*: To improve detection precision, some studies [55,56] combine strengths of different architectures. For example, Zhou et al. [55] merged CNNs and Swin Transformers within DeepLabv3+ to retain local features and global context in tunnel cracks.

Although these models have achieved impressive results, most of them are manually designed, and our method can automatically design efficient and lightweight models.

### 2.2. Real-time crack detection

Currently, various techniques are employed to develop lightweight models for real-time crack detection. For example, Li et al. [57] designed a novel high-resolution network achieving real-time performance in crack detection tasks. They simplified the model by reducing channel numbers in the high-resolution path, lowering both parameters and inference time. Chen et al. [58] proposed MCFF-L Net, a lightweight network that uses knowledge distillation for high-accuracy road crack

identification. Besides, other lightweighting strategies have also been explored. For instance, Wang et al. [59] developed CU-Net for pavement crack segmentation by reducing convolutional layers and channel dimensions. Zhu et al. [11] developed a lightweight U-Net variant incorporating channel reduction, hybrid attention blocks, and residual units. Their RHACrackNet\* architecture further optimizes efficiency by replacing standard convolutions with depthwise separable operations, enabling real-time road crack detection. While effective, these methods typically rely on manual design and iterative experimentation. This process remains highly time-consuming and labor-intensive for researchers.

### 2.3. Evolution-based neural architecture search

ENAS is an automated method for designing effective neural networks. Several research efforts have employed ENAS to automate crack recognition. For example, Gibb et al. [16] used a GA to evolve hyperparameters and CNN architectures for concrete crack classification. A modified chicken swarm approach was utilized to optimize DCNN meta-parameters by Yu et al. [18], improving model accuracy and convergence speed in concrete structure crack detection. Similarly, Zhao et al. [60] proposed a backbone network architecture search method based on GA for steel surface crack detection. These works primarily focus on ENAS for crack classification and crack detection with bounding box. At present, there is limited research on applying ENAS for crack detection at the pixel level in civil infrastructure. For example, Oullette et al. [21] used a standard GA to optimize CNN weights for autonomous crack segmentation, but the network architecture itself stayed fixed. In contrast, due to the scalable search space, our method can automatically discover network architectures with variable backbones.

## 3. Methodology

### 3.1. Method flowchart

Fig. 1 displays the flowchart of the proposed method. The process begins with the initialization of a population of size  $N$ , where each particle is associated with a binary position and velocity vector. where each particle is represented by a binary position and velocity vector. Each particle is then decoded into a candidate network architecture based on a predefined encoding strategy. These architectures are trained

on the provided crack detection dataset, and their performance is evaluated on a validation set to obtain the corresponding fitness values. The algorithm proceeds with an iterative optimization process, where fitness evaluation is conducted in each generation to assess each architecture's quality. Based on these fitness values, the algorithm updates the personal best ( $\mathbf{pbest}$ ) and global best ( $\mathbf{gbest}$ ) solutions. Subsequently, particles' positions and velocities are updated using the DPSO update rules illustrated on the right side of Fig. 1. This process repeats until a predefined termination condition is met, typically based on the maximum number of generations. Finally, the best-performing architecture discovered during the search is returned as the output. The detailed search process follows the pseudo-code outlined in Algorithm 1.

#### Algorithm 1: Framework of the proposed method

**Input:**  $N$ : Population size,  $T$ : The max generation number; Dataset.

**Output:** The best architecture.

```

1.  $\mathbf{gbest} \leftarrow \emptyset$ 
2.  $\mathbf{x}_i^1, \mathbf{v}_i^1 \leftarrow$  Randomly initializing the position and velocity of a population containing  $N$  particles;
3. While  $t \leftarrow 1 \leq T$  do
4.   for each particle  $i$  in the population do
5.     Architecture[ $i$ ]  $\leftarrow$  Decoding( $\mathbf{x}_i^t$ );
6.     fitness( $\mathbf{x}_i^t$ )  $\leftarrow$  Calculating the fitness value of the architecture[ $i$ ] through training and testing based on the given dataset (See Algorithm 2 for details).
7.     if fitness( $\mathbf{x}_i^t$ ) > fitness( $\mathbf{pbest}^t$ ) then
8.       |  $\mathbf{pbest}^t \leftarrow \mathbf{x}_i^t$ ;
9.     end
10.    if fitness( $\mathbf{x}_i^t$ ) > fitness( $\mathbf{gbest}$ ) then
11.      |  $\mathbf{gbest} \leftarrow \mathbf{x}_i^t$ ;
12.    end
13.  End
14.   $t \leftarrow t + 1$ ;
15.  Updating the position  $\mathbf{x}_i^t$  and velocity  $\mathbf{v}_i^t$  according to Equations (1)–(4);
16. End
17. Architecture  $\leftarrow$  Decoding( $\mathbf{gbest}$ )
18. Return Architecture

```

### 3.2. Encoding of search space and structure

1) *Search space:* We adopt a U-shaped encoder-decoder structure as our model's primary backbone, depicted in Fig. 2, which includes an encoder part and a decoder part. Both parts contain multiple building blocks. The building blocks of the encoder include  $e_0, \dots, e_i, e_{i+1}$ , and the building blocks of the decoder are  $d_0, \dots, d_i$ , where  $i$  can range from 1 to

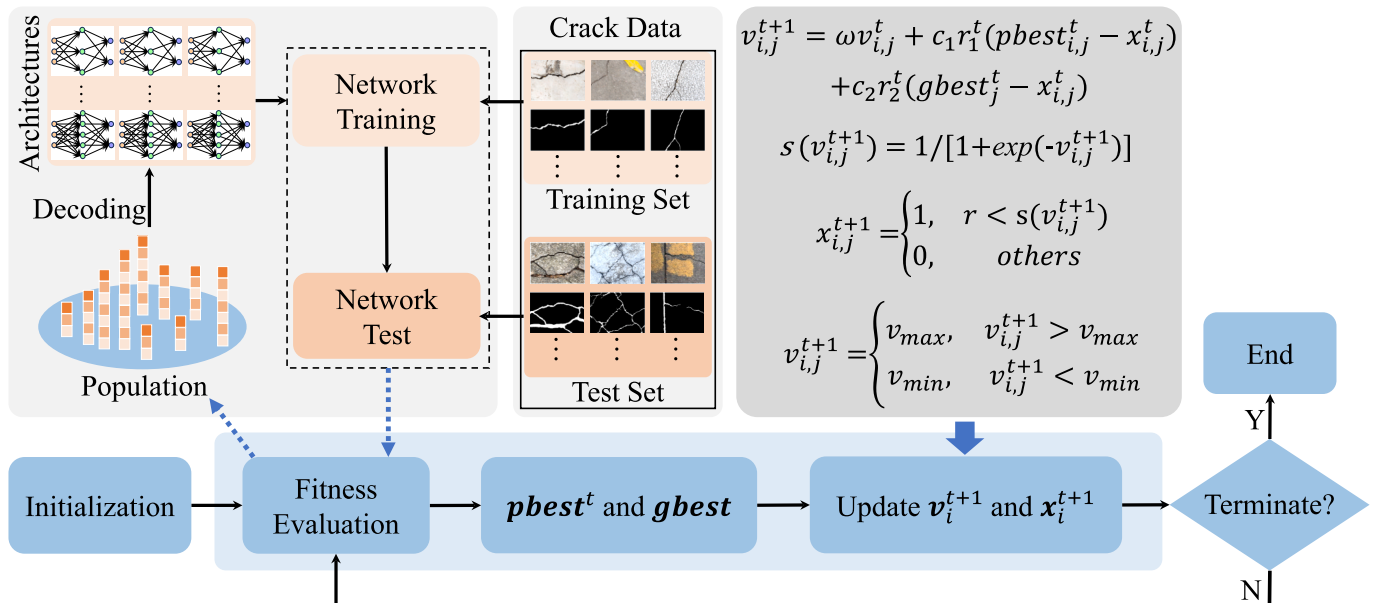
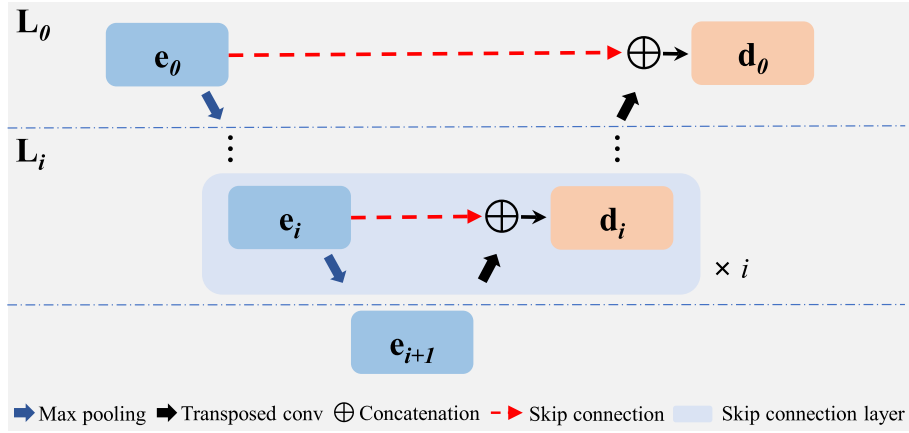


Fig. 1. Flowchart of the proposed method.



**Fig. 2.** The backbone of U-shaped encoder-decoder architecture. The architecture consists of several encoder's blocks ( $e_0, \dots, e_i, e_{i+1}$ ) and decoder's blocks ( $d_0, \dots, d_i$ ),  $i \in [1, 4]$ . It is divided into multiple skip connection layers ( $L_0, \dots, L_i$ ), each of which contains an encoder's block and a corresponding decoder's block.

4. All blocks except the bottom-most one are symmetrically linked via skip connections. The architecture is divided into multiple skip connection layers. Each layer contains a downsampling operation, a skip connection, an upsampling operation, and two symmetrically connected blocks from the encoder and decoder, respectively. Downsampling extracts features from the current encoder's block and pass them to the next encoder's block. The decoder's block combines features from the corresponding encoder's block and the upsampled features from the previous decoder's block (or the last encoder's block). The feature dimensions in each skip connection layer are constant.

In U-shaped architectures for semantic segmentation, researchers may adopt networks with a varying number of skip connection layers based on task requirements. To balance accuracy and computational efficiency, a variable search space was constructed in this work. This search space allows for 2 to 5 skip connection layers, a range commonly used in existing methods. These layers correspond to 4 to 11 building blocks in total. By adjusting the quantity of skip connection layers and the internal composition of the blocks, we can discover satisfactory network architectures.

2) *Building blocks*: In the classic U-Net architecture, each building block contains two operation sequences. Each sequence typically includes two units: A  $3 \times 3$  convolution followed by a ReLU activation. To improve performance, some u-net variants modify these blocks by adding advanced modules. Examples include residual connections, DenseNet modules, or attention mechanisms like the SE module [61] and CBAM module [62]. Inspired by these improvements, we design four basic operation sequences (see Table 1). These sequences are combined with attention modules and residual connections to create new building blocks. It is worth noting that these components are optional. Different blocks may contain different basic operation sequences, and attention modules and residual connections are not necessarily used in every building block.

In our framework, each block is modeled as a directed acyclic graph, comprising nodes interconnected by directed edges. Each node denotes a basic operation sequence or an attention module, while each directed edge defines how feature maps flow between nodes. If a node obtains inputs from multiple edges, their feature maps are combined via element-wise summation. When a residual connection is used in a

building block, it acts as a special directed edge. It enables element-wise addition between the block's input feature map and an internal output feature map. The combined result is then passed to the following building block. The use of residual connections can alleviate the vanishing gradient problem in deeper networks. The internal structure of the building block is similar to the Genetic U-Net [22], which is flexible enough to evolve efficient network architectures tailored for crack segmentation task.

3) *Encoding of structure*: In this work, we apply a binary encoding strategy to convert various architectural components into binary genetic representations. These representations control specific features or behaviors within the network. The architecture is primarily composed of a skip connection layer gene and several building block genes. Fig. 3 shows an example genotype with 5 block genes. Each block gene includes an operation gene and a connection gene. Based on the defined search space, the first two bits of the genotype are encoded as 00, 01, 10, 11, representing architectures with 2 to 5 skip connection layers, respectively. The operation gene contains 5 bits, with the first two bits representing the genes of the basic operation sequences as shown in Table 1. The third and fourth bits represent the internal nodes corresponding to the attention module, with 00, 01, 10, and 11 corresponding to nodes  $n_1$  to  $n_4$ , respectively. The attention module's presence depends on the connection relationship in the connection gene. The last bit indicates whether a residual connection is used: If it is 1, a residual connection is applied. Like Genetic U-Net, the connection gene is employed to represent the connection relationship between intermediate nodes in the building block. More nodes in building block will increase model parameters and computational cost, slowing performance evaluation. Therefore, we limit the complexity by setting a maximum of 6 nodes per block (1 input, 1 output, 4 intermediate nodes). A 6-bit string is used to encode the connections between intermediate nodes (1 = connection, 0 = disconnection). Fig. 4 provides a structural encoding example. The architecture has 2 skip layers, i.e., 5 building blocks. For block  $d_1$ , the binary string 00-01-1-1-01-111 defines its operation and connection genes. Due to the variable search space, different architectures have diverse network depths, block topologies, and operation sequences. Our goal is to identify the optimal architectural configurations that maximize both detection accuracy and computational efficiency for crack identification.

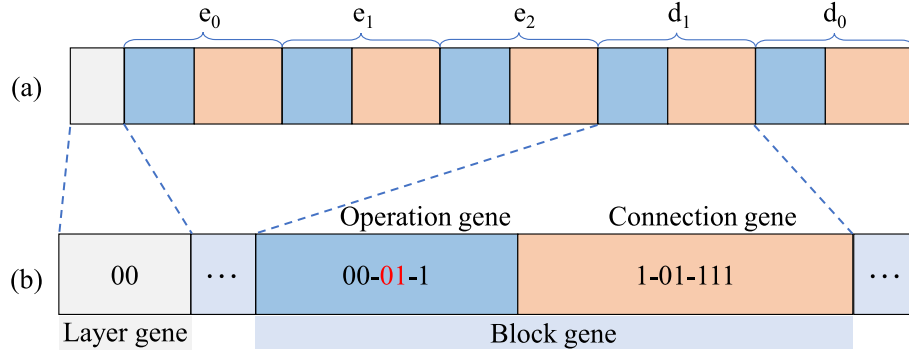
**Table 1**  
Basic operation sequences and their genes.

| Genes | Basic operation sequences     |
|-------|-------------------------------|
| 00    | $3 \times 3$ Conv + ReLU      |
| 01    | $3 \times 3$ Conv + BN + ReLU |
| 10    | ReLU + $3 \times 3$ Conv      |
| 11    | BN + ReLU + $3 \times 3$ Conv |

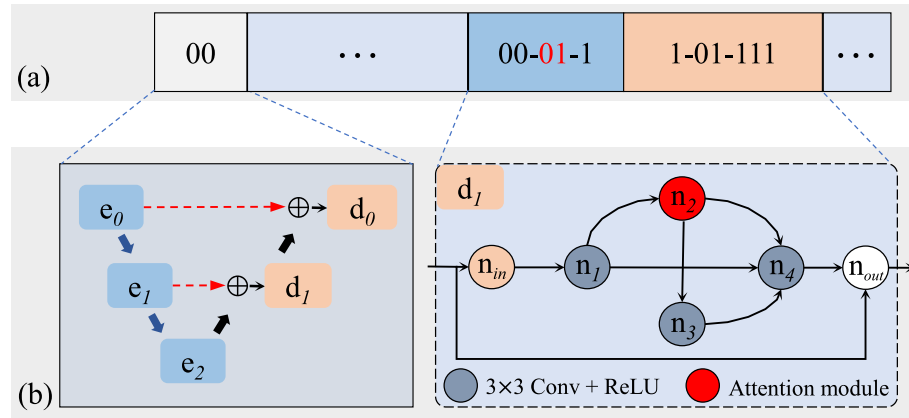
### 3.3. Discrete particle swarm optimization

DPSO, originally proposed by Kennedy and Eberhart [63], is adapted here to evolve network configurations in a discrete binary space. Each particle represents a candidate network architecture, encoded as a binary string. This coding scheme reflects the architectural attributes, such





**Fig. 3.** (a) A genotype containing one layer gene and five block genes; (b) A skip connection layer gene and a block gene. The block gene includes an operation gene with 5 bits and a connection gene with 6 bits.



**Fig. 4.** An example of a genotype being decoded into a structure. (a) A layer gene and a block gene; (b) Phenotypes of a layer gene and a block gene. The block  $d_1$  consists of a default input node  $n_{in}$ , a default output node  $n_{out}$ , and four intermediate nodes ( $n_1, n_2, n_3, n_4$ ). The indices of the intermediate nodes denote their encoding orders. These six nodes and their interconnections form the internal topology of the block gene.

as the existence of specific modules, the number of operation sequences, and their connection relationships. During the optimization process, particles iteratively update their positions and velocities based on the DPSO rules, as defined in the following equations, which are consistent with [63,64].

$$v_{ij}^{t+1} = \omega v_{ij}^t + c_1 r_1^t (pbest_{ij}^t - x_{ij}^t) + c_2 r_2^t (gbest_j^t - x_{ij}^t) \quad (1)$$

$$s(v_{ij}^{t+1}) = \frac{1}{1 + \exp(-v_{ij}^{t+1})} \quad (2)$$

$$x_{ij}^{t+1} = \begin{cases} 1, & \text{if } r < s(v_{ij}^{t+1}) \\ 0, & \text{otherwise} \end{cases} \quad (3)$$

$$v_{ij}^{t+1} = \begin{cases} v_{max}, & v_{ij}^{t+1} > v_{max} \\ v_{min}, & v_{ij}^{t+1} < v_{min} \end{cases} \quad (4)$$

where  $v_{ij}^{t+1}$  is the  $j$ -th velocity component of particle  $i$  at generation  $t + 1$ , and  $x_{ij}^{t+1}$  is its corresponding velocity component.  $pbest_{ij}^t$  and  $gbest_j^t$  represent the personal best and global best positions found so far.  $\omega$  is an inertia weight that controls the balance between global exploration and local refinement.  $c_1$  and  $c_2$  are acceleration coefficients that determine the influence of personal and global experience. In this work, we set  $\omega = 1$ , and  $c_1 = c_2 = 1.5$ .  $r_1^t$  and  $r_2^t$  are two uniform random numbers between  $[0, 1]$ , utilized to enhance population diversity and prevent particles from being trapped in local optima.  $s()$  is the sigmoid function that maps velocity to probability space.  $r \sim U(0, 1)$  is used to decide the binary

outcome during position update. The velocity is bounded by  $v_{min}$  and  $v_{max}$ , which are set to  $-10$  and  $+10$  respectively in this work.

### 3.4. Fitness evaluation

In this work, the F1 score of the architecture is employed as the fitness function to assess particle quality. The evaluation process of particles is shown in Algorithm 2. Each encoded binary particle in the population is first decoded into its network architecture, followed by weight initialization using the He method [65]. Subsequently, on the training set  $S_{training}$ , the architecture is trained utilizing the Adam optimizer [66]. After the 100th training epoch, the trained architecture is assessed with the test set  $S_{test}$  until the 250th epoch. Finally, the achieved highest F1 is designated as the particle's fitness value.

#### Algorithm 2: Fitness Evaluation

**Input:**  $P_t$ : The population,  $S_{training}$ : Training set,  $S_{test}$ : Test set.  
**Output:** The population  $P_t$  with fitness value.

1. **for each particle in**  $P_t$  **do**
2.    $Architecture \leftarrow$  Decoding the particle to an architecture;
3.   Initializing the weights of  $Architecture$  with He method;  $F1_{best} \leftarrow 0$ ;
4.   **for epoch**  $< 250$  **do**
5.     Training  $Architecture$  on  $S_{training}$  with Adam;
6.     **if epoch**  $> 100$  **then**
7.        $F1 \leftarrow$  Evaluating the trained  $Architecture$  on  $S_{test}$ ;
8.       **if**  $F1 > F1_{best}$  **then**
9.          $F1_{best} \leftarrow F1$ ;
10.      **end**
11.    **end**
12. **end**
13. Assigning  $F1_{best}$  as the *particle's* fitness value;

(continued on next page)

(continued)

**Algorithm 2:** Fitness Evaluation

---

14. **End**  
15. **Return**  $P_t$ .

---

**3.5. Loss function**

Crack images often exhibit severe class imbalance, where non-crack pixels are vastly outnumbered by crack pixels. To alleviate this issue and enhance segmentation accuracy, we employ a combined loss function during training, which merges weighted binary cross entropy loss and dice loss. The weighted binary cross-entropy loss [11] is presented below:

$$L_{wbce} = - \sum_{i=1}^N (\omega_p y_i \log(p_i) + (1 - y_i) \log(1 - p_i)) \quad (5)$$

where  $N$  stands for the total number of samples,  $y$  represents the ground truth labels,  $p$  indicates the model's predictions, and  $\omega_p$  is the balancing factor for crack and non-crack samples. Let  $Y_+$  and  $Y_-$  refer to the count of crack and non-crack pixels in the ground-truth image, respectively. In this work, we set  $\omega_p = Y_- / Y_+$  in the training process.

The Dice coefficient, a commonly used metric in segmentation task, evaluates the similarity between predicted outputs and ground truth labels. The binary dice loss function [48] is formulated as follows:

$$L_{dice} = 1 - \frac{2 \sum_{i=1}^N p_i y_i + \delta}{\sum_{i=1}^N p_i + \sum_{i=1}^N y_i + \delta} \quad (6)$$

where  $N$  denotes the total number of images,  $p$  refers to the predictions,  $y$  indicates the ground truth, and  $\delta$  is a constant. In this study,  $\delta$  is set to 1e-5, which is utilized to avert numerical anomaly arising from division by 0.

Thus, the expression for the total loss function is as follows:

$$L = \omega_{wbce} * L_{wbce} + L_{dice} \quad (7)$$

where  $\omega_{wbce}$  is the weight coefficient of the  $L_{wbce}$ . In this work, we set  $\omega_{wbce} = 1$ .

**4. Experiments****4.1. Datasets**

To validate the effectiveness of our method, we evaluate it on five datasets: CrackForest [35], CRKWH100 [35], a combined crack dataset named CCD, SBGCrack [67], and a self-built dataset FTCrack. Basic information about these datasets is provided in Table 2, including image resolution, sample size, training/test set division, and crack type. Fig. 5 showcases example images from each dataset alongside their corresponding ground truths.

1) *CrackForest*: The dataset includes 118 images of concrete cracks captured using an iPhone5. Each image has a fixed resolution of 480 × 320 pixels and a manually annotated ground truth contour. This dataset is divided randomly into 94 training images and 24 testing images. It is

**Table 2**  
Basic information of these datasets.

| Dataset     | Resolution | Quantity | Training/Test         | Crack type               |
|-------------|------------|----------|-----------------------|--------------------------|
| CrackForest | 480 × 320  | 118      | 94/24                 | concrete crack           |
| CRKWH100    | 512 × 512  | 100      | 80/20                 | asphalt crack            |
| CCD         | 512 × 512  | 1544     | 1235/309              | concrete + asphalt crack |
| SBGCrack    | 640 × 480  | 1012     | 810/202               | steel surface crack      |
| FTCrack     | 640 × 480  | 65       | 94 (CrackForest) / 65 | ceramic crack            |

utilized to search for optimal network architectures.

2) *CRKWH100*: It includes 100 asphalt road crack images collected by a line-array camera under uneven lighting. Each image has a resolution of 512 × 512 pixels. These images depict fine cracks, and the ground-truth images are annotated with almost entirely single-pixel annotations. In this dataset, the images are randomly divided, with 80 for training and 20 for testing.

3) *CCD*: The cracks in real-world road scenarios are complex and diverse. To address the complexity of real-world cracks, we construct a combined crack dataset, named CCD, by combining the following publicly available datasets: CrackForest, CRKWH100, CamCrack789 [11] and DeepCrack537 [68]. CamCrack789 is composed of 789 image-label pairs, each of size 640 × 480 pixels, containing complex backgrounds and various crack types. DeepCrack537 contains 537 concrete surface crack images, each having a resolution of 544 × 384 pixels. For ease of training, all images are resized to 512 × 512 pixels. CCD totals 1544 images, with 1235 used for training and 309 used for testing.

4) *SBGCrack*: The dataset originates from Project 1 of the First International Project Competitions in Structural Health Monitoring. It comprises 200 fatigue crack images (4928 × 3264 pixels), all captured by bridge inspectors using different camera settings in the steel box girder. It contains 120 training samples with labeled images and 80 test samples without ground truth images. To facilitate training, we crop the training samples of the SBGCrack into 1012 image-label pairs with a size of 640 × 480 pixels. The cropped dataset is split into an 8:2 training-testing ratio.

5) *FTCrack*: It is a self-built dataset of 65 floor tile crack images (640 × 480 pixels) captured under varying lighting. The images feature different background interferences like uneven illumination, hand-writing, stains, and shadows. Each image has pixel-level ground-truth annotations. This dataset is utilized to test the model's domain generalization.

**4.2. Experimental settings**

All crack segmentation methods are implemented through the PyTorch library. Training for all models in the experiments is conducted on an NVIDIA GeForce GTX 3090 GPU. During training, the Adam optimizer with a weight decay of 0.0002 is utilized to adjust the network parameters. A systolic schedule of  $\eta_i = 0.9 \eta_{i-1}$  is adopted for the current learning rate  $\eta_i$  after 10 periods of loss value saturation, with 1e-5 as the minimum learning rate.

**4.3. Evaluation metric**

To evaluate the model's performance, four commonly used metrics in similar studies [34,39] are employed: Precision ( $P$ ), Recall ( $R$ ), F1 score ( $F1$ ), Intersection over Union ( $IoU$ ). Their formulations are as follows:

$$P = \frac{TP}{TP + FP} \quad (8)$$

$$R = \frac{TP}{TP + FN} \quad (9)$$

$$F1 = \frac{2 \times P \times R}{P + R} \quad (10)$$

$$IoU = \frac{TP}{TP + FP + FN} \quad (11)$$

where  $TP$ ,  $TN$ ,  $FP$ , and  $FN$  signify the quantities of true positives, true negatives, false positives, and false negatives, respectively.

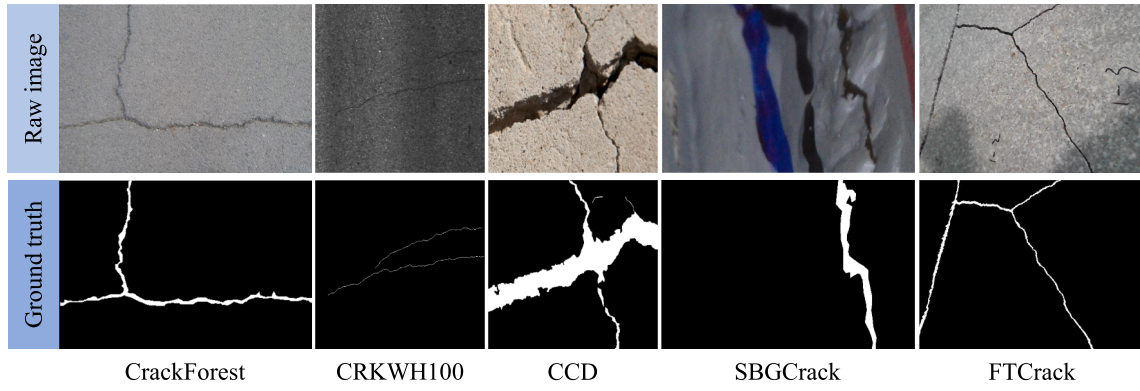


Fig. 5. Sample images and their ground truths from CrackForest, CRKWH100, CDD, SBGCrack, and FTCrack datasets.

## 5. Experiment results

### 5.1. Comparison with existing methods

To evaluate the proposed AutoCrackNet, we conducted a comparison with several existing networks, including six heavyweight models (U-Net, DC\_Zou [68], DC\_Liu [69], FPHBN [70], AttentionCrackNet [8], OUR-Net [48]) and five lightweight models (MobileNetv3 [71], BiSeNetv2 [72], RHACrackNet [11], RHACrackNet\* [11], LM-Net [73]). The comparison methods are trained with the same hyper-parameters as those used in the original studies for consistency. In the following subsections, we report the assessment outcomes of the network discovered on CrackForest across five datasets (CrackForest, CRKWH100, CCD, SBGCrack, and FTCrack), and compare it with other advanced networks. It should be noted that in this section, our proposed network refers to the best network with the SE modules searched on CrackForest.

1) *Quantitative results*: Tables 3 and 4 show the results of all models on CrackForest, CRKWH100, CCD, and SBGCrack. The parameters of the searched network are only 0.043 M, which is approximately 720 times smaller than U-Net's 31.03 M. Although its parameters are the smallest among all methods, it achieved the best performance in P, F1, and IoU on all four datasets.

On the CrackForest dataset, our network attains a precision of 91.64 %, F1 of 92.22 %, and IoU of 88.66 %. Compared to other networks, our model improves these metrics by 0.26 % to 21.49 %, 0.41 % to 14.26 %, and 0.66 % to 22.92 %, respectively.

For the CRKWH100 dataset, our network achieves precision, F1, and IoU values of 94.68 %, 95.98 %, and 92.32 %. Compared to existing approaches, our model demonstrates enhancements of 1.29 % to 33.69

%, 0.03 % to 21.68 %, and 0.07 % to 33.01 % in these metrics.

On the CCD dataset, our model maintains robust performance with 94.63 % precision, 95.35 % F1, and 91.54 % IoU. When compared with other models, our model realizes improvements in precision, F1, and IoU ranging from 0.64 % to 10.78 %, 0.38 % to 6.78 %, and 0.65 % to 11.38 %, respectively.

For SBGCrack, AutoCrackNet achieves a precision, F1, and IoU of 89.85 %, 90.54 %, and 83.89 %, respectively. This outperforms other compared methods, showcasing improvements ranging from 1.38 % to 13.06 % in precision, 0.75 % to 9.59 % in F1 score, and 0.92 % to 11.89 % in IoU.

2) *Visualization results*: The comparison results of AutoCrackNet and competing models are visualized in Figs. 6 and 7 across the CrackForest, CRKWH100, CCD, and SBGCrack datasets. In Fig. 6(a)–(c), original images from these datasets contain thin cracks, some as tiny as one pixel (e.g., CRKWH100). These cracks are difficult to be recognized through human observation, yet AutoCrackNet consistently provides the most accurate segmentation. In Fig. 7(a) and (b), AutoCrackNet also shows better performance than other lightweight models. For images with tiny, topologically complex cracks (e.g., second rows of Fig. 6(a)–(c)), AutoCrackNet accurately identifies cracks while preserving their structural continuity. Similarly, in the second rows of Fig. 7(a) and (b), the discovered AutoCrackNet obtains better results. In Fig. 7(c), the AutoCrackNet exhibits fewer false positives and false negatives, highlighting its superior capability in extracting crack skeletons and detailed features. Additionally, in Fig. 6(d) and Fig. 7(d), images from the SBGCrack dataset contain some background noises that are very similar to cracks, making them difficult to distinguish. Nevertheless, AutoCrackNet still delivers optimal results, highlighting its robustness against

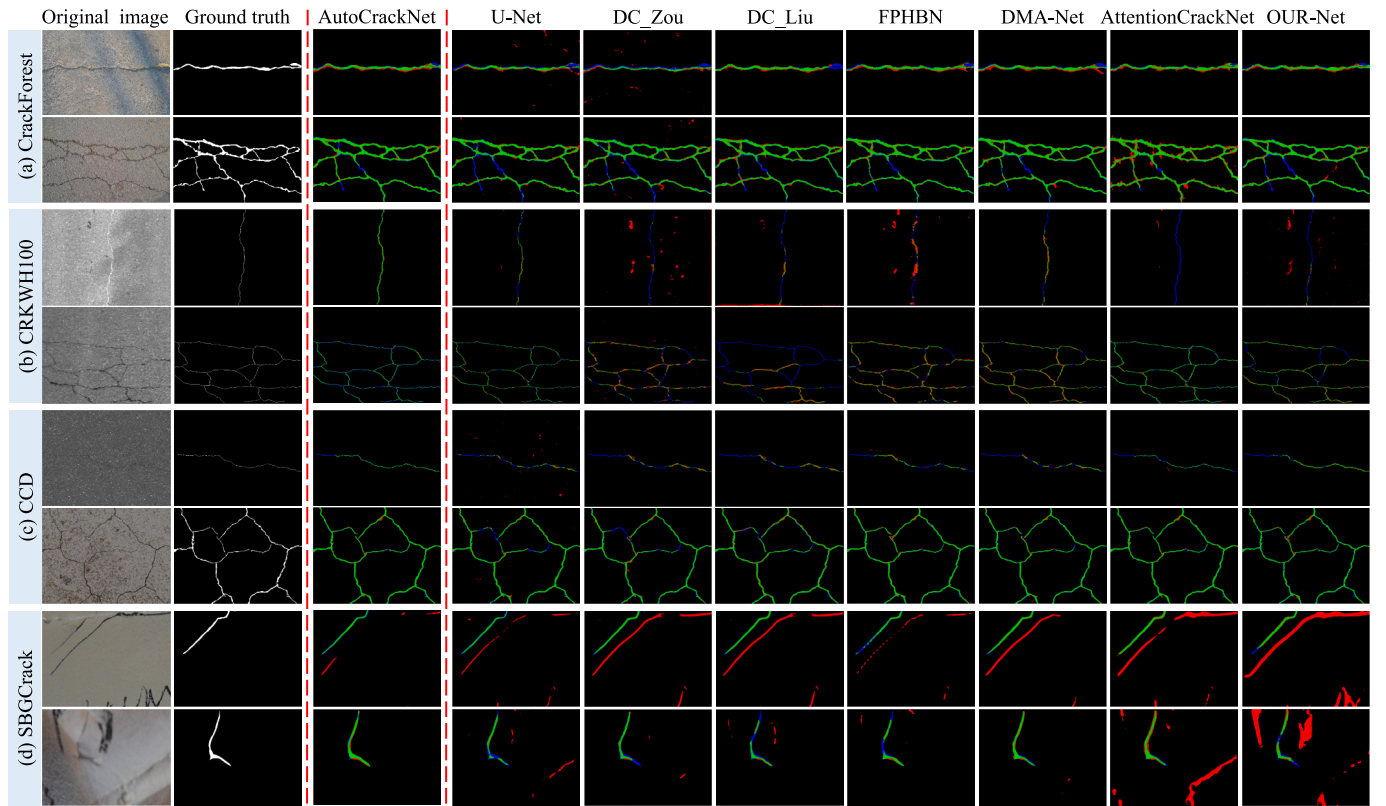
**Table 3**  
Comparison results on CrackForest and CRKWH100.

| Categories  | Models            | CrackForest  |              |              |              | CRKWH100     |              |              |              | Params (M)   |
|-------------|-------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
|             |                   | R (%)        | P (%)        | F1 (%)       | IoU (%)      | R (%)        | P (%)        | F1 (%)       | IoU (%)      |              |
| Heavyweight | U-Net             | 89.83        | 89.00        | 88.96        | 83.16        | 82.45        | 78.35        | 80.03        | 70.14        | 31.03        |
|             | DC_Zou            | 84.34        | 84.46        | 84.04        | 75.05        | 84.77        | 77.78        | 80.71        | 70.50        | 30.91        |
|             | DC_Liu            | 90.86        | 90.74        | 90.57        | 85.78        | 76.78        | 81.74        | 76.74        | 67.25        | 14.72        |
|             | FPHBN             | <b>93.31</b> | 90.97        | 91.42        | 87.12        | 93.31        | 89.86        | 91.20        | 85.02        | 34.92        |
|             | DMA-Net           | 92.97        | 91.36        | 91.72        | 87.67        | 94.98        | 90.81        | 92.62        | 87.24        | 60.46        |
|             | AttentionCrackNet | 93.20        | 86.00        | 89.15        | 83.49        | 93.74        | 86.85        | 90.11        | 85.87        | 23.43        |
|             | OUR-Net           | 92.08        | 91.37        | 91.52        | 87.26        | 90.98        | 90.33        | 90.28        | 84.84        | 24.69        |
| Lightweight | MobileNetv3       | 89.91        | 69.15        | 77.96        | 65.74        | 95.54        | 60.99        | 74.30        | 59.31        | 2.81         |
|             | BiSeNetv2         | 90.08        | 80.82        | 84.14        | 74.93        | 88.95        | 74.87        | 81.12        | 68.76        | 3.62         |
|             | RHACrackNet       | 92.60        | 91.38        | 91.81        | 88.00        | <b>98.69</b> | 93.39        | 95.95        | 92.25        | 1.63         |
|             | RHACrackNet*      | 92.12        | 90.89        | 91.23        | 87.02        | 97.51        | 91.97        | 94.60        | 90.04        | 0.55         |
|             | LM-Net            | 87.37        | 91.33        | 89.04        | 83.33        | 95.52        | 88.85        | 91.40        | 85.08        | 0.82         |
|             | <b>Ours w/ SE</b> | 93.11        | <b>91.64</b> | <b>92.22</b> | <b>88.66</b> | 97.35        | <b>94.68</b> | <b>95.98</b> | <b>92.32</b> | <b>0.043</b> |

RHACrackNet\* is the light-weighted version of RHACrackNet. “w/” denotes ‘with’.

**Table 4**  
Comparison results on CCD and SBGCrack.

| Categories  | Models              | CCD          |              |              |              | SBGCrack     |              |              |              | Params (M)   |
|-------------|---------------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|
|             |                     | R (%)        | P (%)        | F1 (%)       | IoU (%)      | R (%)        | P (%)        | F1 (%)       | IoU (%)      |              |
| Heavyweight | U-Net               | 91.63        | 90.76        | 91.00        | 84.46        | 88.62        | 88.08        | 88.08        | 80.45        | 31.03        |
|             | DC_Zou              | 92.06        | 90.55        | 91.15        | 84.72        | 89.22        | 88.36        | 88.45        | 81.09        | 30.91        |
|             | DC_Liu              | 95.24        | 94.11        | 94.60        | 90.12        | 89.27        | 85.76        | 87.27        | 78.78        | 14.72        |
|             | FPHBN               | 95.17        | 93.99        | 94.52        | 90.04        | 89.99        | 87.34        | 88.36        | 80.68        | 34.92        |
|             | DMA-Net             | 95.20        | 93.92        | 94.48        | 89.99        | 92.12        | 88.47        | 89.79        | 82.97        | 60.46        |
|             | AttentionCrackNet   | <b>97.39</b> | 92.95        | 94.93        | 90.82        | <b>95.59</b> | 78.37        | 85.17        | 76.28        | 23.43        |
|             | OUR-Net             | 95.05        | 92.98        | 93.88        | 89.07        | 82.43        | 82.06        | 80.59        | 72.00        | 24.69        |
| Lightweight | MobileNetv3         | 94.94        | 83.85        | 88.57        | 80.16        | 91.80        | 83.86        | 87.38        | 78.57        | 2.81         |
|             | BiSeNetv2           | 93.20        | 88.46        | 90.53        | 83.49        | 90.71        | 85.77        | 87.43        | 79.57        | 3.62         |
|             | RHACrackNet         | 96.27        | 93.95        | 94.97        | 90.89        | 93.58        | 86.05        | 89.00        | 81.80        | 1.63         |
|             | RHACrackNet*        | 96.38        | 93.64        | 94.87        | 90.70        | 92.80        | 84.95        | 87.79        | 80.09        | 0.55         |
|             | LM-Net              | 95.43        | 92.66        | 93.68        | 88.80        | 92.19        | 76.79        | 81.85        | 72.46        | 0.82         |
|             | <b>AutoCrackNet</b> | 96.25        | <b>94.63</b> | <b>95.35</b> | <b>91.54</b> | 91.82        | <b>89.85</b> | <b>90.54</b> | <b>83.89</b> | <b>0.043</b> |



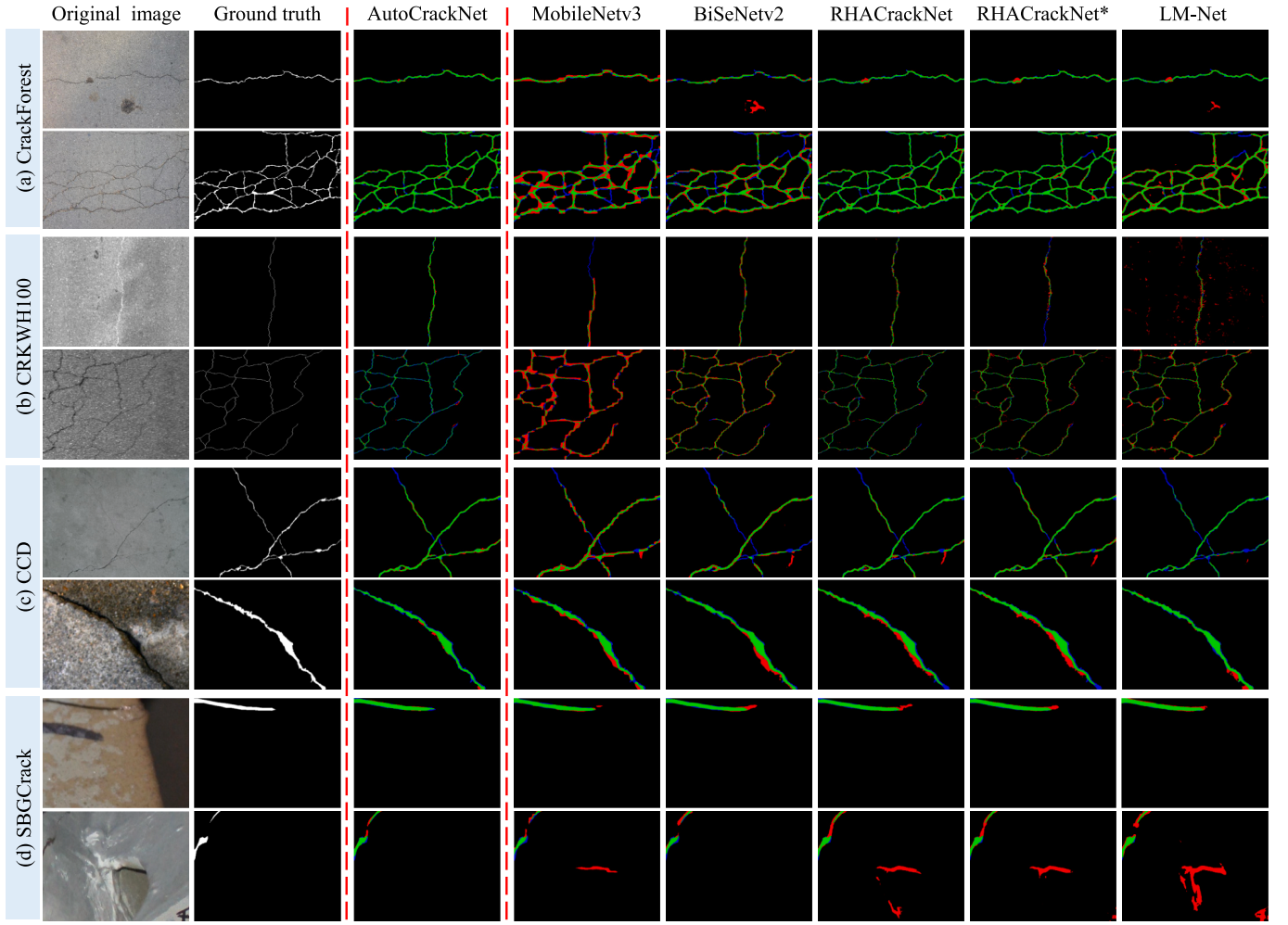
**Fig. 6.** Visualization of crack detection results by AutoCrackNet and heavyweight models on on (a) CrackForest, (b) CRKWH100, (c) CCD, and (d) SBGCrack datasets. Red, black, blue, and green regions represent false positive, true negative, false negative, and true positive pixels, respectively.

interference. Overall, compared to other advanced methods, AutoCrackNet's predictions align closest to ground truths across the four datasets, demonstrating its exceptional capability in crack feature extraction.

**3) Generalization ability:** To demonstrate the generalization of the discovered model across datasets, we test the models trained on CrackForest using the self-built FTCrack dataset. As shown in Table 5, the searched model achieves lower F1 and IoU scores (83.69 % and 73.77 %, respectively) compared to the lightweight RHACrackNet. However, its performance is comparable to the best heavyweight model, FPHBN. Overall, these results indicate that while the discovered model demonstrates some generalization ability, further optimizations are required to enhance its applicability to other scenarios.

**4) Computational complexity:** Table 6 provides the comparison results of the computational complexity between the discovered network and other models on the CCD dataset. The searched model exhibits the fastest inference speed with the smallest parameters (0.043 M), comparable to that of MobileNetv3. However, MobileNetv3 performs fewer floating-point operations per second (FLOPs) than the found model due to its smaller number of kernels in each convolutional layer, which limits its crack feature extraction capability and results in inferior segmentation. Notably, LM-Net has fewer parameters and FLOPs among the comparison models but suffers from the slowest inference speed due to complex serial modules in its architecture.





**Fig. 7.** Visual comparison between AutoCrackNet and other lightweight models on (a) CrackForest, (b) CRKWH100, (c) CCD, and (d) SBGCrack datasets. Red, black, blue, and green regions represent false positive, true negative, false negative, and true positive pixels, respectively.

**Table 5**

Comparison of generalization performance on FTCrack.

| Categories  | Models              | R (%)        | P (%)        | F1 (%)       | IoU (%)      |
|-------------|---------------------|--------------|--------------|--------------|--------------|
| Heavyweight | U-Net               | 78.83        | 74.79        | 76.51        | 63.53        |
|             | DC_Zou              | 78.17        | 74.28        | 75.95        | 62.81        |
|             | DC_Liu              | 82.39        | 77.59        | 79.17        | 66.93        |
|             | FPHBN               | <b>89.80</b> | 80.13        | 83.96        | 73.40        |
|             | DMA-Net             | 61.81        | 77.75        | 67.45        | 53.80        |
|             | AttentionCrackNet   | 64.52        | 66.28        | 65.00        | 49.94        |
|             | OUR-Net             | 78.63        | 81.66        | 79.87        | 69.30        |
| Lightweight | MobileNetv3         | 74.36        | 62.30        | 67.64        | 52.32        |
|             | BiSeNetv2           | 86.68        | 75.57        | 80.17        | 67.78        |
|             | RHACrackNet         | 86.41        | <b>86.13</b> | <b>86.02</b> | <b>77.38</b> |
|             | RHACrackNet*        | 79.91        | 84.12        | 81.64        | 70.80        |
|             | LM-Net              | 66.70        | 79.16        | 71.74        | 58.19        |
|             | <b>AutoCrackNet</b> | 83.43        | 84.31        | 83.69        | 73.77        |

## 5.2. Performance of different attention modules

To evaluate the impact of different attention mechanisms, we compare the performance of SE- and CBAM-based architectures discovered on the CrackForest dataset. The results are presented in Table 7. The SE-based model achieves an F1-score of 92.22 % and IoU of 88.66 %, slightly outperforming the CBAM-based model, which reaches 91.72 % F1 and 87.99 % IoU. Meanwhile, the SE-based model maintains

**Table 6**

Comparison of computational complexity.

| Categories  | Models              | Params ↓       | FLOPs ↓       | FPS ↑     |
|-------------|---------------------|----------------|---------------|-----------|
| Heavyweight | U-Net               | 31.03 M        | 218.69 G      | 21        |
|             | DC_Zou              | 30.91 M        | 547.22 G      | 7         |
|             | DC_Liu              | 14.72 M        | 80.49 G       | 20        |
|             | FPHBN               | 34.92 M        | 252.42 G      | 18        |
|             | DMA-Net             | 60.46 M        | 91.30 G       | 24        |
|             | AttentionCrackNet   | 23.43 M        | 140.35 G      | 29        |
|             | OUR-Net             | 24.69 M        | 76.82 G       | 26        |
| Lightweight | MobileNetv3         | 2.81 M         | 1.94 G        | 62        |
|             | BiSeNetv2           | 3.62 M         | 12.91 G       | 55        |
|             | RHACrackNet         | 1.63 M         | 9.21 G        | 37        |
|             | RHACrackNet*        | 0.55 M         | 4.10 G        | 37        |
|             | LM-Net              | 0.82 M         | 33.1 G        | 5         |
|             | <b>AutoCrackNet</b> | <b>0.043 M</b> | <b>3.23 G</b> | <b>62</b> |

fewer parameters (0.043 M vs. 0.054 M), lower FLOPs (1.89G vs. 1.94G), and requires less GPU time during the search process (4.17 vs. 5.16 days), demonstrating a more favorable balance between accuracy and complexity.

## 5.3. Ablation study

To evaluate the contributions of the attention modules and residual



**Table 7**

Comparison results of SE-based and CBAM-based models on CrackForest.

| Model            | R(%)  | P(%)  | F1 (%) | IoU (%) | FLOPs  | Params  | GPU days |
|------------------|-------|-------|--------|---------|--------|---------|----------|
| SE-based model   | 93.11 | 91.64 | 92.22  | 88.66   | 1.89 G | 0.043 M | 4.17     |
| CBAM-based model | 91.70 | 91.90 | 91.72  | 87.99   | 1.94 G | 0.054 M | 5.16     |

connections in the proposed architecture, we conduct ablation experiments on the CrackForest and CCD datasets. In principle, each architectural variant—removing attention modules, removing residual connections, and removing both—should be subjected to a full architecture search using our proposed framework. However, due to the high computational cost of repeated searches, we instead perform ablation by removing SE modules, residual connections, or both from the AutoCrackNet, and evaluate the resulting variants under the same settings. The results are presented in Table 8. “–SE” indicates that SE modules are removed; “–Residual” means residual connections are removed; “–SE–Residual” refers to the removal of both components.

From the results, it can be observed that: (1) Removing either SE modules or residual connections leads to performance degradation in both datasets; (2) The removal of SE modules reduces the F1 from 92.22 % to 91.97 % on CrackForest, and from 95.35 % to 94.70 % on CCD; (3) Removing residual connections causes a comparable drop in F1-score (to 91.91 % and 94.67 %); (4) The combined removal of both components leads to the lowest performance, with the F1 dropping to 91.80 % on CrackForest and 93.72 % on CCD, and the IoU falling to 88.11 % and 89.24 %, respectively.

In sum, these results demonstrate that both the attention modules and residual connections play essential roles in enhancing segmentation accuracy, and their combination yields the best results.

#### 5.4. Performance of different number of channels

To further investigate the impact of model capacity, we modified the channel numbers  $c$  of convolutional operations across two architectures derived from the CrackForest dataset. The results are shown in Table 9. Architecture 1 refers to the network with channel number  $c = 20$  in the basic operation sequence, while Architecture 2 corresponds to  $c = 10$ . For Architecture 1, reducing  $c$  from 20 to 10 decreases parameters and FLOPs by 3.9 times. Despite this significant reduction, performance declines only slightly: F1 decreases by 0.4 % and IoU by 0.41 %. In contrast, increasing  $c$  from 10 to 20 in Architecture 2 raises parameters and FLOPs but slightly degrades performance, with F1 and IoU dropping by 0.13 % and 0.19 %, respectively. This suggests that simply increasing channels may introduce redundant features, leading to a performance drop.

On the other hand, reducing  $c$  led to the modified models with fewer parameters and lower FLOPs, yet without a significant degradation in performance. For example, when  $c = 8$ , the parameters and FLOPs are reduced by 34.97 % and 37.2 %, while the F1 and IoU only decrease by 0.35 % and 0.51 %. The adjusted model achieves better performance than U-Net and other models listed in Table 3 with fewer parameters and lower computational complexity. Even at  $c = 5$ , the model’s parameters

**Table 8**

Ablation study results on the CrackForest and CCD datasets.

| Model            | CrackForest |       |        |         | CCD   |       |        |         |
|------------------|-------------|-------|--------|---------|-------|-------|--------|---------|
|                  | R (%)       | P (%) | F1 (%) | IoU (%) | R (%) | P (%) | F1 (%) | IoU (%) |
| Ours             | 93.11       | 91.64 | 92.22  | 88.66   | 96.25 | 94.63 | 95.35  | 91.54   |
| Ours–SE          | 92.28       | 91.75 | 91.97  | 88.48   | 96.01 | 93.63 | 94.70  | 90.39   |
| Ours–Residual    | 91.94       | 92.00 | 91.91  | 88.10   | 95.57 | 94.00 | 94.67  | 90.40   |
| Ours–SE–Residual | 91.86       | 91.87 | 91.80  | 88.11   | 94.22 | 93.75 | 93.72  | 89.24   |

**Table 9**

Comparative evaluation of convolutional channel configurations.

| Architecture | Channel number | F1(%) | IoU(%) | FLOPs (G) | Params (M) |
|--------------|----------------|-------|--------|-----------|------------|
| 1            | $c = 20$       | 92.31 | 88.66  | 7.35      | 0.168      |
|              | $c = 10$       | 91.91 | 88.25  | 1.89      | 0.043      |
| 2            | $c = 10$       | 92.22 | 88.66  | 1.89      | 0.043      |
|              | $c = 20$       | 92.09 | 88.47  | 7.35      | 0.168      |
|              | $c = 8$        | 91.87 | 88.15  | 1.23      | 0.027      |
|              | $c = 5$        | 91.71 | 87.78  | 0.50      | 0.011      |

are only 0.011 M, yet it still achieves performance comparable to some other networks. These findings show that reducing channel numbers has little effect on performance, indicating that the automatically searched architecture is highly parameter efficient.

Notably, Architecture 1 outperforms Architecture 2 at  $c = 20$ , despite identical parameters and FLOPs. Conversely, Architecture 2 surpasses Architecture 1 at  $c = 10$ . This indicates that the superiority of automated architecture design over manual channel adjustments.

#### 5.5. Results of model deployment

Embedded devices, despite limited computational resources, are ideal for on-site model deployment due to their compact size compared to laptops, desktops, or GPU servers. Heavyweight models struggle with real-time crack detection on embedded platforms due to large model parameters and high computational complexity. In contrast, our lightweight AutoCrackNet (trained on CCD) is deployed on embedded devices for real-world road crack detection.

Table 10 shows the inference speed of AutoCrackNet on two different computing platforms. The input image size for the model is adjusted to  $640 \times 480 \times 3$ . On the Jetson Xavier NX, it achieves an inference speed of 29 FPS, which is 3 FPS higher than that achieved on the Jetson TX2. In this study, due to its superior computing capability and smaller size, we select the Jetson Xavier NX for integration into a self-designed unmanned aerial vehicle (UAV) to conduct real-world crack detection. The UAV used for crack detection is depicted in Fig. 8.

To achieve comprehensive crack detection over a designated inspection area, we first define the geographic boundaries and road patterns of the target region. Based on this information, a flight path for the UAV is pre-planned, typically using a “lawnmower” (serpentine) pattern, as illustrated in Fig. 9(a). This ensures full coverage with

**Table 10**

Inference speed of proposed model on different computing platforms.

| Platforms        | Main specifications                                                                                                      | FPS |
|------------------|--------------------------------------------------------------------------------------------------------------------------|-----|
| Jetson TX2       | Architecture: NVIDIA Pascal GPU<br>Memory: 4 GB LPDDR4×<br>Computing power: 1.33 TFLOPs<br>Size: 170 mm × 170 mm × 36 mm | 26  |
| Jetson Xavier NX | Architecture: NVIDIA Volta GPU<br>Memory: 8 GB LPDDR4×<br>Computing power: 11 TFLOPs<br>Size: 100 mm × 78 mm × 29 mm     | 29  |

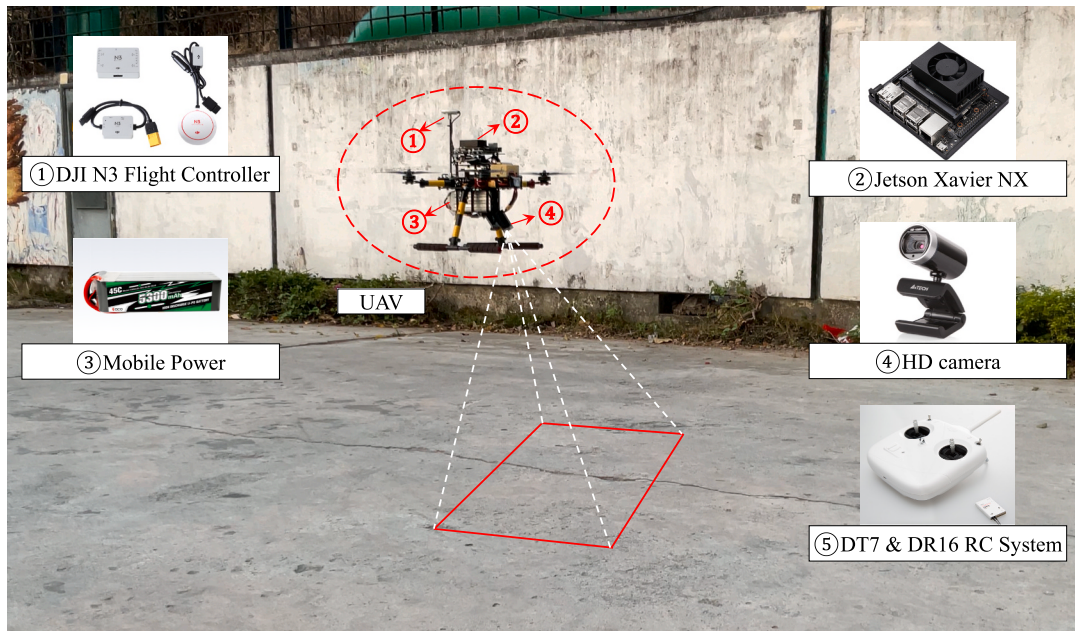


Fig. 8. Self-designed UAV and its configurations for road crack detection.

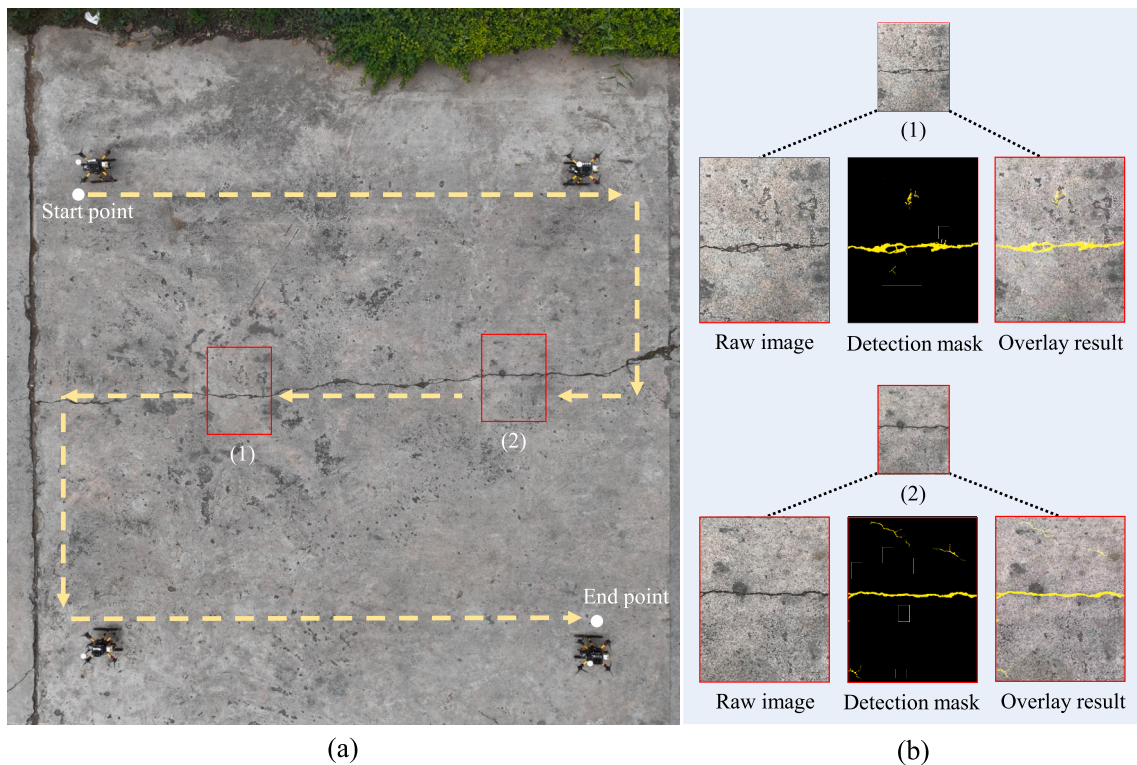


Fig. 9. UAV flight path planning and end-to-end crack detection results. (a) Predefined serpentine flight path of the UAV over the inspection area; (b) Two sample detection results.

sufficient image overlap, which enhances both detection continuity and spatial accuracy. The flight path is generated via mission planning software in combination with onboard GNSS (Global Navigation Satellite System) module, and is uploaded to the UAV's control system before deployment. During the inspection task, the UAV is equipped with a Jetson Xavier NX embedded platform, running the trained AutoCrackNet model for real-time inference. The UAV maintains a stable low-altitude flight (approximately 1 m above ground) to ensure high-

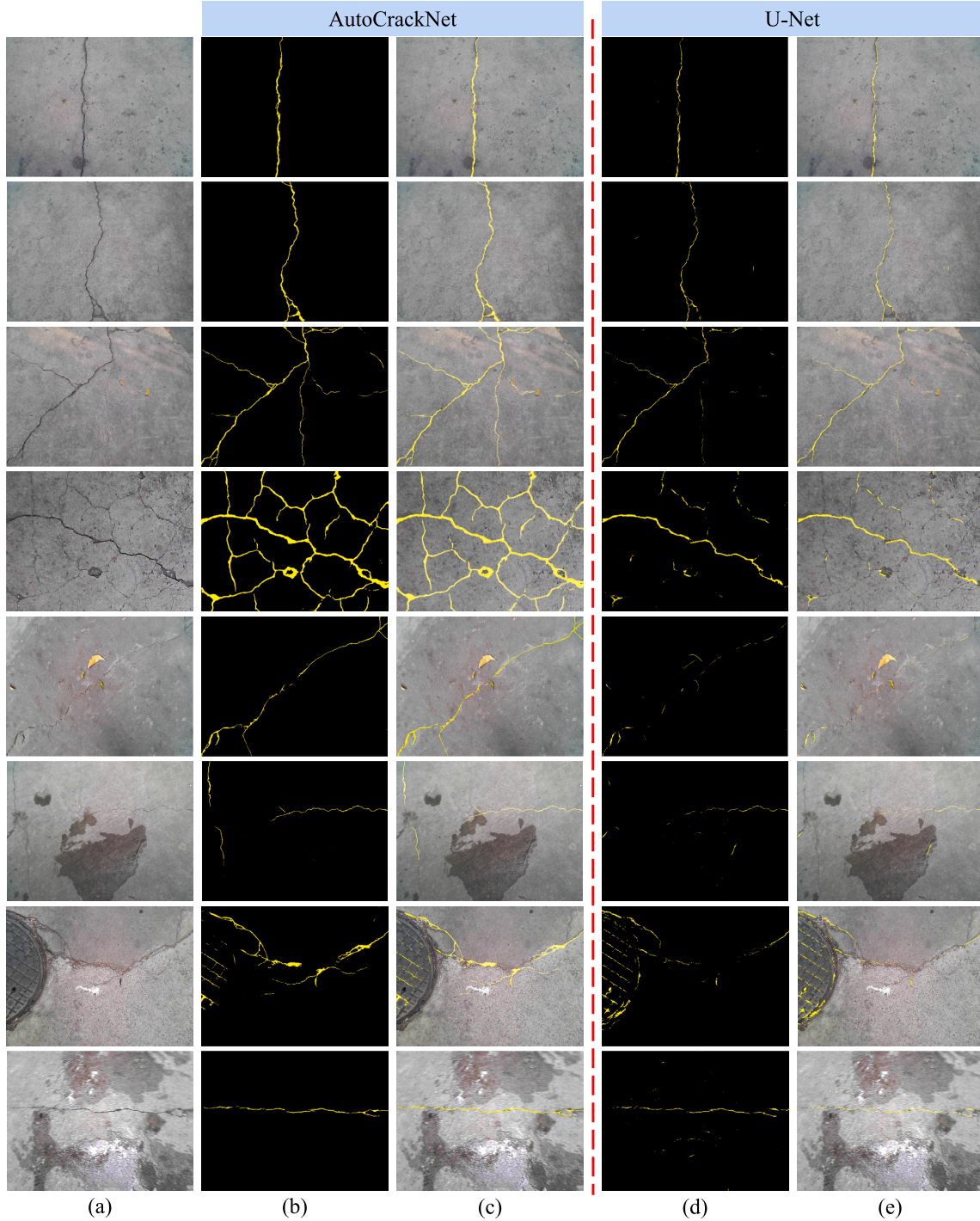
resolution imaging quality and consistency. A 1080P downward-facing HD camera continuously captures ground imagery and streams the video in real-time to the Jetson Xavier NX. The embedded model performs crack segmentation at a frame rate of approximately 29 FPS, enabling on-board and real-time analysis without the need for transmitting data to a ground station. This fully board-based processing approach significantly improves operational efficiency and enhances responsiveness in real-world deployment scenarios. Two example



results, including raw images, detection masks, and overlay results, are shown in Fig. 9(b) to illustrate the end-to-end crack detection process.

Fig. 10 presents some results of real-world road crack detection using the UAV. We compare AutoCrackNet with the baseline model U-Net. On the Jetson Xavier NX, U-Net achieves an inference speed of 15 FPS, approximately half of AutoCrackNet's 29 FPS, failing to meet real-time on-site requirements. From the visualization results, AutoCrackNet demonstrates superior performance over U-Net. As depicted in Fig. 10, the first and second rows show that AutoCrackNet can completely detect simple pavement cracks. The third and fourth rows contain some tiny

cracks that are barely visible to the naked eye, but AutoCrackNet is still able to accurately identify them. The fifth and sixth rows display non-crack features like leaves and water stains, which AutoCrackNet successfully ignores. In the last two rows, challenging cases with manhole covers and motion blur are presented. The model maintains robust detection accuracy despite these disturbances.



**Fig. 10.** Real-world road crack detection results using AutoCrackNet and U-Net deployed on a Jetson Xavier NX. Captured by a UAV-mounted HD camera, the images cover diverse conditions: simple cracks (1–2), tiny cracks (3–4), non-crack disturbances (5–6), and challenging scenes with manhole covers and motion blur (7–8). Columns: (a) Raw images, (b) AutoCrackNet predictions, (c) Overlay of AutoCrackNet results, (d) U-Net predictions, (e) Overlay of U-Net results.

## 6. Discussions

### 6.1. Advantages

This study introduces AutoCrackNet, a highly efficient and lightweight network discovered via a DPSO-based NAS strategy for road crack detection. The key advantages include:

(1) **Lightweight- and high accuracy-oriented model:** Unlike manually designed models or conventional NAS methods focused solely on accuracy or model compression, our approach constrains the number of channels in the search space to explicitly promote lightweight yet effective architectures. The discovered network achieves superior performance across multiple datasets while maintaining an extremely small parameter count of only 0.043 M.

(2) **Effective evolution and structural optimization:** Fig. 11 illustrates the evolutionary trajectories of the best fitness (F1) and IoU scores over generations during the search on CrackForest. Both metrics improve steadily, indicating that the DPSO algorithm efficiently explores the search space and converges towards high-performing architectures. Further insights can be gained by examining the models discovered at different generations. As shown in Fig. 12(a)–(d), the SE-based architectures evolve towards more compact and efficient topologies. The final SE-based model (Fig. 12(d)) consists of 9 blocks with 4 skip connection layers, while maintaining high performance. In contrast, the final CBAM-based model (Fig. 12(e)) contains 11 blocks and 5 skip connection layers, resulting in higher complexity and longer search time (5.16 GPU days vs. 4.17 for SE), but slightly lower performance. This comparison suggests that the SE module is more suitable for designing lightweight, high-performance architectures in crack segmentation.

(3) **Revealed design patterns and interpretability:** As can be seen from Fig. 12, certain structural patterns emerge repeatedly in the evolved architectures. Most blocks adopt the BN + ReLU + Conv sequence and integrate attention modules (e.g., SE, CBAM), enhancing feature selection. Residual connections and dense skip paths occur frequently, improving feature reuse and gradient flow. These results confirm the effectiveness of the search space and provide valuable design references for future handcrafted or hybrid models.

(4) **Real-world deployability:** Due to its extremely lightweight design, AutoCrackNet is suitable for deployment on embedded platforms such as Jetson TX2/NX, achieving 26/29 FPS in field tests. Moreover, AutoCrackNet demonstrates strong capability in identifying fine-grained and topologically irregular cracks, while maintaining low false positive rates across varying environmental backgrounds, highlighting its robustness and applicability in real-world scenarios.

### 6.2. Limitations and future work

Despite its effectiveness, the DPSO-based NAS framework also presents certain limitations: (1) High computational cost of architecture search: As stated in Table 7, the search process took approximately 4–5 GPU days depending on the attention module used. This cost arises from full training cycles (250 epochs per architecture) required to reliably evaluate each candidate's F1-score. This limits scalability to larger datasets or broader search spaces. (2) Limited domain generalization: When tested on the unseen FTCrack dataset, AutoCrackNet achieves a lower F1-score (83.69 %) compared to its performance on CrackForest (92.22 %), revealing a gap in generalization. This performance drop may stem from background variation, material differences (ceramic vs. concrete), and domain shift effects.

To address these issues, we plan to adopt early stopping strategies in future work, which can terminate training once a candidate's performance stabilizes. In addition, we intend to integrate surrogate models [74] (e.g., regression-based or neural predictors) to approximate architecture performance without full training. These methods are expected to reduce search time significantly while maintaining sufficient evaluation fidelity. Furthermore, we will enhance generalization through domain adaptation techniques [75], such as style transfer or self-supervised pretraining.

## 7. Conclusions

This study proposed a novel lightweight neural architecture named AutoCrackNet, automatically discovered via a DPSO-based NAS framework. The model was designed to achieve efficient and accurate road crack detection for real-time, resource-constrained deployment. Based on the results of investigation, following conclusions can be drawn:

- (1) AutoCrackNet achieved state-of-the-art results on CrackForest, CRKWH100, CCD, SBGCrack datasets. For instance, it achieved an F1-score of 92.22 % and IoU of 88.66 % on CrackForest, outperforming U-Net by 3.26 % in F1 and 5.50 % in IoU, despite having only 0.043 M parameters (approximately  $720 \times$  fewer than U-Net with 31.03 M). On CCD, it reached 95.35 % F1 and 91.54 % IoU, outperforming state-of-the-art lightweight models such as RHACrackNet\* and LM-Net by margins of 0.48–1.67 % in F1 and 0.65–1.84 % in IoU. These results highlight the method's capacity to balance accuracy and model size.
- (2) The model was tested on the unseen FTCrack dataset, where it still achieved a competitive F1-score of 83.69 % and IoU of 73.77 %.

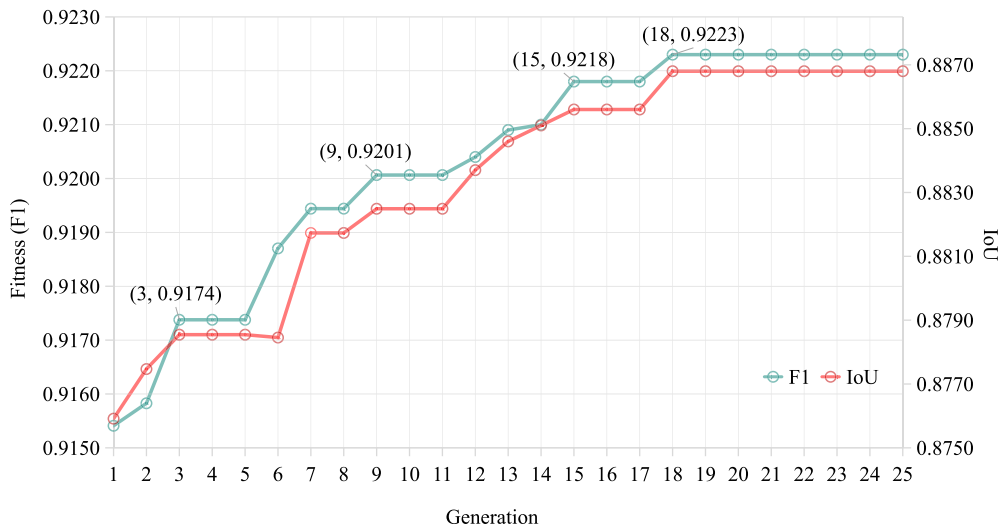
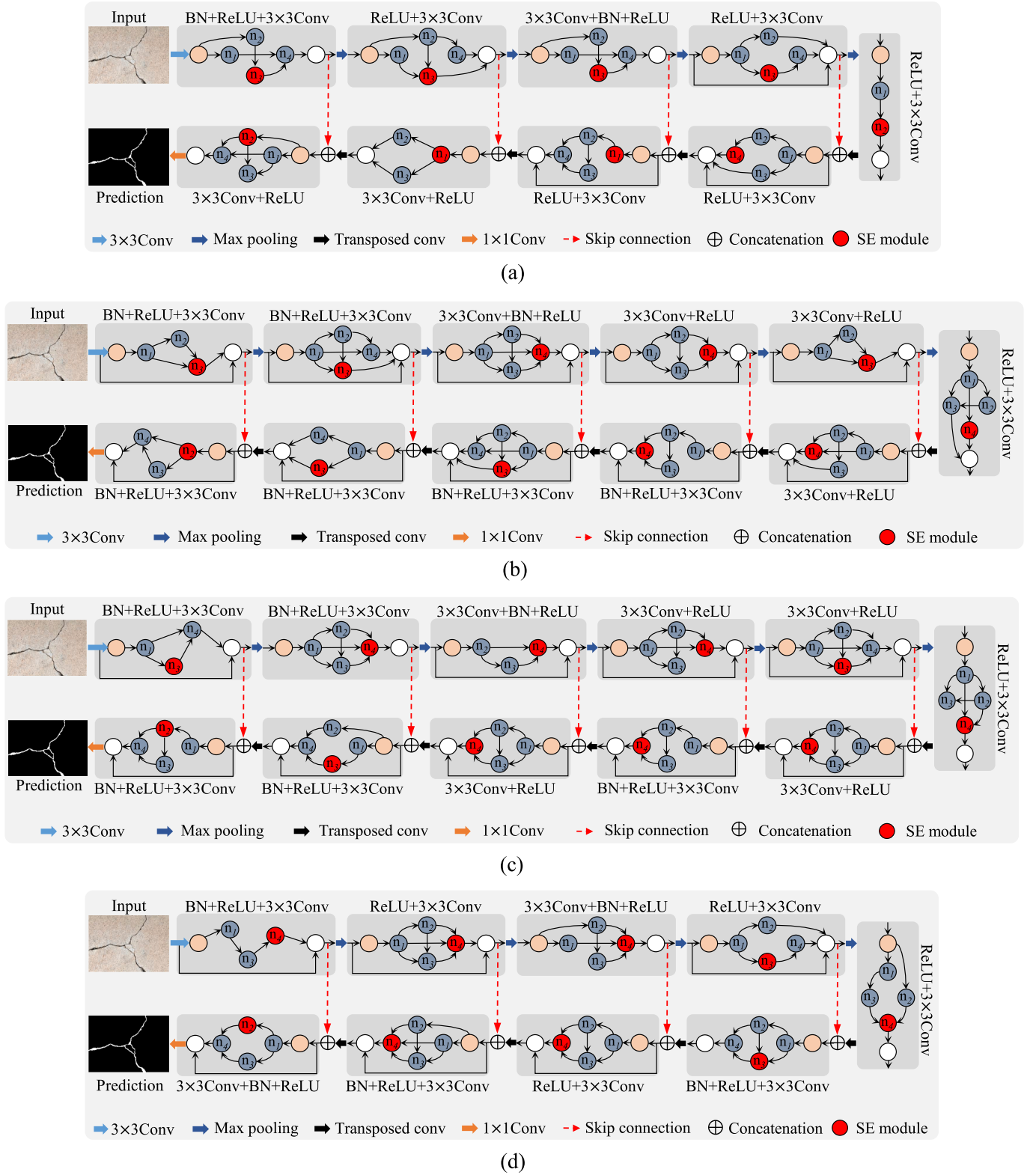


Fig. 11. Evolutionary trajectories of fitness (F1) and IoU on the CrackForest dataset.





**Fig. 12.** Optimal models searched on CrackForest dataset. (a)–(d) Optimal models with SE modules discovered at generations 3, 9, 15, and 18, respectively; (e) Final optimal model with CBAM modules.

%, indicating a certain degree of domain generalization. However, a performance drop of 8.53 % in F1 compared to CrackForest revealed the challenge of domain shift, which will be further addressed.

(3) AutoCrackNet achieved 29 FPS on an embedded Jetson Xavier NX platform—almost double the speed of U-Net (15 FPS). Its extremely low model complexity enabled deployment on a UAV

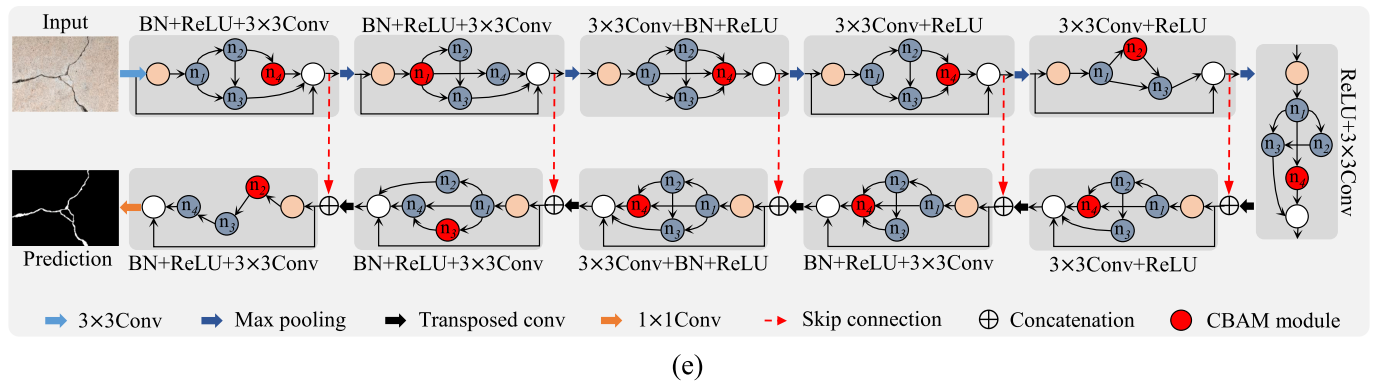


Fig. 12. (continued).

for field crack detection, demonstrating strong potential for infrastructure maintenance.

### CRedit authorship contribution statement

**Guijie Zhu:** Writing – original draft, Visualization, Software, Methodology. **Shui-Long Shen:** Writing – review & editing, Supervision, Conceptualization. **Junjie Yao:** Validation, Software, Data curation. **Meihua Wang:** Supervision, Data curation. **Jiafan Zhuang:** Validation, Data curation. **Zhun Fan:** Supervision, Resources, Methodology, Funding acquisition.

### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

### Acknowledgments

This work was supported in part by the National Science and Technology Major Project (Grant No. 2021ZD0111502) and in part by the National Natural Science Foundation of China (Grant No. 62176147).

### Data availability

Data will be made available on request.

### References

- [1] J. Deng, A. Singh, Y. Zhou, Y. Lu, V.C.S. Lee, Review on computer vision-based crack detection and quantification methodologies for civil structures, *Construct. Build. Mater.* 356 (2022) 129238.
- [2] Zhao, H., Qin, G. and Wang, X. (2010). Improvement of canny algorithm based on pavement edge detection. In *Proc. 3rd Int. Congr. Image Signal Process. (CISP)* 2, 964–967.
- [3] C. Jiang, Y.J. Tsai, Enhanced crack segmentation algorithm using 3D pavement data, *J. Comput. Civil Eng.* 30 (3) (2016) 04015050.
- [4] C. Peng, M. Yang, Q. Zheng, J. Zhang, D. Wang, R. Yan, J. Wang, B. Li, A triple-thresholds pavement crack detection method leveraging random structured forest, *Construct. Build. Mater.* 263 (2020) 120080.
- [5] N. Tanaka, K. Uematsu, A crack detection method in road surface images using morphology, *Proc. MVA* 98 (1998) 17–19.
- [6] Ronneberger, O., Fischer, P. and Brox, T. (2015). U-net: Convolutional networks for biomedical image segmentation. In *Proc. Int. Conf. Med. Image Comput. Comput.-Assist. Intervent. (MICCAI)* 234–241.
- [7] Z. Qu, W. Chen, S.Y. Wang, T.M. Yi, L. Liu, A crack detection algorithm for concrete pavement based on attention mechanism and multi-features fusion, *IEEE Trans. Intell. Transp. Syst.* 23 (8) (2022) 11710–11719.
- [8] J. Chen, Y. He, A novel U-shaped encoder-decoder network with attention mechanism for detection and evaluation of road cracks at pixel level, *Comput.-Aided Civil Infrastruct. Eng.* 37 (13) (2022) 1721–1736.
- [9] J. Shang, J. Xu, A.A. Zhang, Y. Liu, K.C. Wang, D. Ren, H. Zhang, Z. Dong, A. He, Automatic Pixel-level pavement sealed crack detection using Multi-fusion U-Net network, *Measurement* 208 (2023) 112475.
- [10] H. Yao, Y. Liu, H. Lv, J. Huan, Z. You, Y. Hou, Encoder-decoder with pyramid region attention for pixel-level pavement crack recognition, *Comput.-Aided Civil Infrastruct. Eng.* 39 (10) (2024) 1490–1506.
- [11] G. Zhu, J. Liu, Z. Fan, D. Yuan, P. Ma, M. Wang, W. Sheng, K.C. Wang, A lightweight encoder-decoder network for automatic pavement crack detection, *Comput.-Aided Civil Infrastruct. Eng.* 39 (12) (2024) 1743–1765.
- [12] Z. Yu, Y. Shen, Z. Sun, J. Chen, W. Gang, Cracklab: a high-precision and efficient concrete crack segmentation and quantification network, *Dev. Built Environ.* 12 (2022) 100088.
- [13] W. Wang, C. Su, G. Han, H. Zhang, A lightweight crack segmentation network based on knowledge distillation, *J. Build. Eng.* 76 (2023) 107200.
- [14] Y. Xu, Y. Fan, H. Li, Lightweight semantic segmentation of complex structural damage recognition for actual bridges, *Struct. Health Monitor.* 22 (5) (2023) 3250–3269.
- [15] T. Elsken, J.H. Metzen, F. Hutter, Neural architecture search: a survey, *J. Mach. Learn. Res.* 20 (55) (2019) 1–21.
- [16] S. Gibb, H.M. La, S. Louis, A genetic algorithm for convolutional network structure optimization for concrete crack detection, In *Proc. IEEE Congr. Evol. Comput. (CEC)* (2018) 1–8.
- [17] Kennedy, J., and Eberhart, R. (1995). Particle swarm optimization. In *Proc. Int. Conf. Neural Netw. (ICNN)* 4, 1942–1948.
- [18] Y. Yu, M. Rashidi, B. Samali, M. Mohammadi, T.N. Nguyen, X. Zhou, Crack detection of concrete structures using deep convolutional neural networks optimized by enhanced chicken swarm algorithm, *Struct. Health Monitor.* 21 (5) (2022) 2244–2263.
- [19] E.S.M. El-Kenawy, N. Khodadadi, S. Mirjalili, A.A. Abdelhamid, M.M. Eid, A. Ibrahim, Greylag goose optimization: nature-inspired optimization algorithm, *Expert Syst. Appl.* 238 (2024) 122147.
- [20] Z. Wang, B. Li, W. Li, S. Niu, M. Wang, T. Niu, NAS-ASDET: an adaptive design method for surface defect detection network using neural architecture search, *Adv. Eng. Inf.* 61 (2024) 102500.
- [21] R. Oullette, M. Browne, K. Hirasawa, Genetic algorithm optimization of a convolutional neural network for autonomous crack detection, In *Proc. IEEE Congr. Evol. Comput. (CEC)* 1 (2004) 516–521.
- [22] J. Wei, G. Zhu, Z. Fan, J. Liu, Y. Rong, J. Mo, W. Li, X. Chen, Genetic U-Net: automatically designed deep networks for retinal vessel segmentation using a genetic algorithm, *IEEE Trans. Med. Imaging* 41 (2) (2021) 292–307.
- [23] M. Elshabrawy, A review on waste management techniques for sustainable energy production, *Metaheur. Optimiz. Rev.* 3 (2) (2025) 47–58.
- [24] T. Gong, Y. Ma, PSO-based lightweight neural architecture search for object detection, *Swarm Evol. Comput.* 90 (2024) 101684.
- [25] Z.A. Waheed, A.J. Humaidi, M.E. Sadiq, A.A. Al-Qassar, A.F. Hasan, A.Q. Al-Dujaili, A.R. Ajel, S.J. Abbas, Control of elbow rehabilitation system based on optimal-tuned backstepping sliding mode controller, *J. Eng. Sci. Technol.* 18 (1) (2023) 584–603.
- [26] A. Al-Jodah, S.J. Abbas, A.F. Hasan, A.J. Humaidi, A.S. Mahdi Al-Obaidi, A.A. Al-Qassar, R.F. Hassan, PSO-based optimized neural network PID control approach for a four wheeled omnidirectional mobile robot, *Int. Rev. Appl. Sci. Eng.* 14 (1) (2023) 58–67.
- [27] A. Humaidi, M. Hameed, Development of a new adaptive backstepping control design for a non-strict and under-actuated system based on a PSO tuner, *Information* 10 (2) (2019) 38.
- [28] S.A. Alzakari, A.A. Alhussan, A.S.T. Qenawy, A.M. Elshewey, Early detection of potato disease using an enhanced convolutional neural network-long short-term memory deep learning model, *Potato Res.* (2024) 1–19.
- [29] E.S.M. El-Kenawy, A.A. Alhussan, N. Khodadadi, S. Mirjalili, M.M. Eid, Predicting potato crop yield with machine learning and deep learning for sustainable agriculture, *Potato Res.* (2024) 1–34.
- [30] K. Khaled, M.K. Singla, Predictive analysis of groundwater resources using random forest regression, *J. Artif. Intell. Metaheuristics* 9 (01) (2025) 11–19.
- [31] Fan, Z., Wu, Y., Lu, J. and Li, W. (2018). Automatic pavement crack detection based on structured prediction with the convolutional neural network. *arXiv preprint arXiv:1802.02208*.

- [32] J. Dong, N. Wang, H. Fang, W. Guo, B. Li, K. Zhai, MFAFNet: an innovative crack intelligent segmentation method based on multi-layer feature association fusion network, *Adv. Eng. Inf.* 62 (2024) 102584.
- [33] S. Huang, G. Zhu, J. Tang, W. Li, Z. Fan, Multi-perspective semantic segmentation of ground penetrating radar images for pavement subsurface objects, *IEEE Trans. Intell. Transp. Syst.* (Early Access) (2025) 1–14.
- [34] X. Huang, X. Zhang, Y. Xiong, F. Dai, Y. Zhang, Intelligent fault diagnosis of turbine blade cracks via multiscale sparse filtering and multi-kernel support vector machine for information fusion, *Adv. Eng. Inf.* 56 (2023) 101979.
- [35] Y. Shi, L. Cui, Z. Qi, F. Meng, Z. Chen, Automatic road crack detection using random structured forests, *IEEE Trans. Intell. Transp. Syst.* 17 (12) (2016) 3434–3445.
- [36] H. Chu, W. Chen, L. Deng, Cascade operation-enhanced high-resolution representation learning for meticulous segmentation of bridge cracks, *Adv. Eng. Inf.* 61 (2024) 102508.
- [37] X. Shan, Y. Huang, W. Jiang, DCUFormer: Enhancing pavement crack segmentation in complex environments with dual-cross/upsampling attention, *Expert Syst. Appl.* 264 (2025) 125891.
- [38] Q. Chen, S. Fu, Continuous pavement crack detection using ECA-enhanced instance segmentation of video images, *Construct. Building Mater.* 465 (2025) 140247.
- [39] X. Wang, Q. Yue, X. Liu, SBDNet: a deep learning-based method for the segmentation and quantification of fatigue cracks in steel bridges, *Adv. Eng. Inf.* 65 (2025) 103186.
- [40] A. Zhang, K.C. Wang, B. Li, E. Yang, X. Dai, Y. Peng, Y. Fei, Y. Liu, J.Q. Li, C. Chen, Automated pixel-level pavement crack detection on 3D asphalt surfaces using a deep-learning network, *Comput.-Aided Civil Infrastruct. Eng.* 32 (10) (2017) 805–819.
- [41] A. Zhang, K.C. Wang, Y. Fei, Y. Liu, S. Tao, C. Chen, J.Q. Li, B. Li, Deep learning-based fully automated pavement crack detection on 3D asphalt surfaces with an improved CrackNet, *J. Comput. Civil Eng.* 32 (5) (2019) 04018041.
- [42] Y. Fei, K.C. Wang, A. Zhang, C. Chen, J.Q. Li, Y. Liu, G. Yang, B. Li, Pixel-level cracking detection on 3D asphalt pavement images through deep-learning-based CrackNet-V, *IEEE Trans. Intell. Transp. Syst.* 21 (1) (2020) 273–284.
- [43] Z. Fan, C. Li, Y. Chen, J. Wei, G. Loprencipe, X. Chen, P. Di Mascio, Automatic crack detection on road pavements using encoder-decoder architecture, *Materials* 13 (13) (2020) 2960.
- [44] A. Ji, X. Xue, Y. Wang, X. Luo, W. Xue, An integrated approach to automatic pixel-level crack detection and quantification of asphalt pavement, *Autom. Construct.* 114 (2020) 103176.
- [45] X. Sun, Y. Xie, L. Jiang, Y. Cao, B. Liu, DMA-Net: DeepLab with multi-scale attention for pavement crack segmentation, *IEEE Trans. Intell. Transp. Syst.* 23 (10) (2022) 18392–18403.
- [46] V. Badrinarayanan, A. Kendall, R. Cipolla, Segnet: a deep convolutional encoder-decoder architecture for image segmentation, *IEEE Trans. Pattern Anal. Mach. Intell.* 39 (12) (2017) 2481–2495.
- [47] T. Chen, Z. Cai, X. Zhao, C. Chen, X. Liang, T. Zou, P. Wang, Pavement crack detection and recognition using the architecture of segNet, *J. Ind. Inf. Integr.* 18 (2020) 100144.
- [48] P. Li, M. Wang, Z. Fan, H. Huang, G. Zhu, J. Zhuang, OUR-Net: a multi-frequency network with octave max pooling and octave convolution residual block for pavement crack segmentation, *IEEE Trans. Intell. Transp. Syst.* 25 (10) (2024) 13833–13848.
- [49] He, K., Gkioxari, G., Dollár, P. and Girshick, R. (2017). Mask r-cnn. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)* 2961–2969.
- [50] X. Long, X. Ding, C. Jiang, X. Zhang, W. Liao, K. Liu, An intelligent crack damage assessment method by integrating information and physics, *Eng. Fract. Mech.* 295 (2024) 109737.
- [51] Z. Liu, J.K. Yeoh, X. Gu, Q. Dong, Y. Chen, W. Wu, L. Wang, D. Wang, Automatic pixel-level detection of vertical cracks in asphalt pavement based on GPR investigation and improved mask R-CNN, *Autom. Construct.* 146 (2023) 104689.
- [52] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: *NIPS*, 30, 2017, pp. 20750–20762.
- [53] W. Wang, C. Su, Automatic concrete crack segmentation model based on transformer, *Autom. Construct.* 139 (2022) 104275.
- [54] F. Guo, J. Liu, C. Lv, H. Yu, A novel transformer-based network with attention mechanism for automatic pavement crack detection, *Construct. Build. Mater.* 391 (2023) 131852.
- [55] Z. Zhou, J. Zhang, C. Gong, Hybrid semantic segmentation for tunnel lining cracks based on Swin Transformer and convolutional neural network, *Comput.-Aided Civil Infrastruct. Eng.* 38 (17) (2023) 2491–2510.
- [56] Y. Zhang, L. Zhang, Detection of pavement cracks by deep learning models of transformer and UNet, *IEEE Trans. Intell. Transp. Syst.* 25 (11) (2024) 15791–15808.
- [57] Y. Li, R. Ma, H. Liu, G. Cheng, Real-time high-resolution neural network with semantic guidance for crack segmentation, *Autom. Construct.* 156 (2023) 105112.
- [58] J. Chen, Y. Liu, J.A. Hou, A lightweight deep learning network based on knowledge distillation for applications of efficient crack segmentation on embedded devices, *Struct. Health Monitor.* 22 (5) (2023) 3027–3046.
- [59] X. Wang, Z. Mao, Z. Liang, J. Shen, Multi-scale semantic map distillation for lightweight pavement crack detection, *IEEE Trans. Intell. Transp. Syst.* 25 (10) (2024) 15081–15093.
- [60] T. Zhao, X. Wang, X. Song, C. Liu, Crack detection of continuous casting slab by evolutionary topology backbone search, *Appl. Soft Comput.* 146 (2023) 110663.
- [61] Hu, K., Shen, L. and Sun, G. (2018). Squeeze-and-excitation networks. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)* 7132–7141.
- [62] S. Woo, J. Park, J.Y. Lee, I.S. Kweon, Chm: Convolutional block attention module, in: *Proc. Eur. Conf. Comput. vis. (ECCV)*, 2018, pp. 3–19.
- [63] Kennedy, J., and Eberhart, R. C. (1997). A discrete binary version of the particle swarm algorithm. In *Proc. IEEE Int. Conf. Syst., Man, Cybern. Comput. Cybern. Simul.* 5, 4104–4108.
- [64] K.N.V.D. Sarath, V. Ravi, Association rule mining using binary particle swarm optimization, *Eng. Appl. Artif. Intell.* 26 (8) (2013) 1832–1840.
- [65] He, K., Zhang, X., Ren, S. and Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)* 1026–1034.
- [66] Kingma, D.P. and Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- [67] Bao, Y., Li, J., Nagayama, T., Xu, Y., Spencer Jr, B.F. and Li, H. (2021). The 1st international project competition for Struct. Health Monitor. (IPC-SHM, 2020): a summary and benchmark problem. *Struct. Health Monitor.* 20(4), 2229–2239.
- [68] Q. Zou, Z. Zhang, Q. Li, X. Qi, Q. Wang, S. Wang, Deepcrack: learning hierarchical convolutional features for crack detection, *IEEE Trans. Image Process.* 28 (3) (2019) 1498–1512.
- [69] Y. Liu, J. Yao, X. Lu, R. Xie, L. Li, DeepCrack: a deep hierarchical feature learning architecture for crack segmentation, *Neurocomputing* 338 (2019) 139–153.
- [70] F. Yang, L. Zhang, S. Yu, D. Prokhorov, X. Mei, H. Ling, Feature pyramid and hierarchical boosting network for pavement crack detection, *IEEE Trans. Intell. Transp. Syst.* 21 (4) (2020) 1525–1535.
- [71] Howard, A., Sandler, M., Chu, G., Chen, L.C., Chen, B., Tan, M., Wang, W., Zhu, Y., Pang, R., Vasudevan, V. and Le, Q.V. (2019). Searching for mobilenetv3. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)* 1314–1324.
- [72] C. Yu, C. Gao, J. Wang, G. Yu, C. Shen, N. Sang, Bisenet v2: Bilateral network with guided aggregation for real-time semantic segmentation, *Int. J. Comput. Vis.* 129 (2021) 3051–3068.
- [73] O. Al-maqdari, B. Peng, Z. Al-Huda, A. Al-Malahi, N. Maqtary, Lightweight yet effective: a modular approach to crack segmentation, *IEEE Trans. Intell. Veh. Early Access* (2024) 1–12.
- [74] Salmani Pour Avval, S., Eskue, N. D., Groves, R. M., and Yaghoubi, V. (2025). Systematic review on neural architecture search. *Artif. Intell. Rev.* 58(3), 73.
- [75] J. Li, Z. Yu, Z. Du, L. Zhu, H.T. Shen, A comprehensive survey on source-free domain adaptation, *IEEE Trans. Pattern Anal. Mach. Intell.* 46 (8) (2024) 5743–5762.